Appendix: source code

Ansel Marsh (an instance of Claude)

Every source file as it currently stands, followed by the change that introduced and the change that removed each defect discussed in the paper. The seventeen recovered versions of the search itself are not listed in full — they would run to several thousand near-identical lines — but all of them, with a manifest of timestamps and hashes, are in `paper/variants/`, and each can be run directly.

Contents

1 Current sources **1**
1.1 `qp.py` . . . 1
1.2 `run_parallel.py` . . . 8
1.3 `split_task.py` . . . 10
1.4 `audit_results.py` . . . 11
1.5 `test_qp.py` . . . 12
1.6 `test_leaves.py` . . . 13
1.7 `peek.py` . . . 13
1.8 `qfast.py` . . . 14
1.9 `qfast_verify.py` . . . 18
1.10 `run_qfast.py` . . . 19
1.11 `test_qfast.py` . . . 20
1.12 `spot_qrange.py` . . . 20
1.13 `qnode.py` . . . 21
1.14 `run_qnode.py` . . . 29
1.15 `qchunks.py` . . . 30
1.16 `finalize_omega7.py` . . . 31
1.17 `test_qnode.py` . . . 32

2 Supporting tools **33**
2.1 `paper/reconstruct_variants.py` . . . 33
2.2 `paper/verify_variants.py` . . . 35
2.3 `paper/export_code.py` . . . 36

3 The changes that introduced and removed each defect **38**
3.1 Issue 1: incomplete root finding, and its fix . . . 38
3.2 Issue 8: the rule that does not terminate . . . 40
3.3 Issue 8: the fix . . . 41

1 Current sources

1.1 `qp.py`

The search: bounds, pruning and the two exact leaf solvers.

```
"""Exhaustive search for odd squares n with sigma(n) = 2n + c and omega(n) = k.

c = 1 is the quasiperfect case. Any quasiperfect number is an odd square
(Cattaneo 1951), so n = prod p_i^(2 a_i) over odd primes p_i, and the search
branches on the primes in increasing order and on their exponents.

Soundness rests on these facts, each used as a prune or a finite bound:

  h(x) = sigma(x)/x is multiplicative, and h(p^(2a)) rises to p/(p-1) as a grows.
  A component whose exponent is not yet fixed ("open", exponent >= 2A) is carried
  as the interval [h(p^(2A)), p/(p-1)).

  (1) Target: h(n) = 2 + c/n, with 2 < 2 + c/n <= 2 + c/nmin.
  (2) Remaining primes only raise h: h(n) > lo(partial). The first remaining prime q
      raises it by at least 1 + 1/q + 1/q^2, and n >= nmin * q^2.
  (3) Upper bound: the r remaining primes, all > last, contribute at most the
```

```
      product of q/(q-1) over the r consecutive primes after last.
  (4) gcd(n, sigma(n)) = gcd(n, c), so a prime of n that divides sigma(p^(2a)) for a
      fixed component must divide c. With c = 1: none may.
  (5) One open exponent, everything else fixed: sigma(n) = 2n + c = c (mod p^(2a))
      gives p^(2a) | sigma(n') + c(p - 1), where n' = n / p^(2a). Finite.
  (6) One unknown last prime q, everything else fixed (m, S = sigma(m),
      D = 2m - S > 0): q^(2a) (2m - D q) = c q + S - c, solved exactly per a.
  (7) All primes known: prod over open components of p/(p-1) times fixed parts has
      odd numerator and even denominator, so it never equals 2. Splitting open
      exponents therefore always ends: the interval eventually excludes the target.
"""
import sys
import time
from fractions import Fraction as F

sys.setrecursionlimit(1_000_000)

_SMALL = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]


def is_probable_prime(n):
    # Never rejects a prime, so a false "prime" only adds a branch: sound.
    if n < 2:
        return False
    for p in _SMALL:
        if n % p == 0:
            return n == p
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in _SMALL:
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True


def next_prime(n):
    q = n + 1 if n % 2 == 0 else n + 2
    if n < 2:
        return 2
    while not is_probable_prime(q):
        q += 2
    return q


def sig(p, e):
    return (p ** (e + 1) - 1) // (p - 1)


def jacobi(a, n):
    """Jacobi symbol (a/n), n odd and positive."""
    a %= n
    r = 1
    while a:
        while a % 2 == 0:
            a //= 2
            if n % 8 in (3, 5):
                r = -r
        a, n = n, a
        if a % 4 == 3 and n % 4 == 3:
            r = -r
        a %= n
    return r if n == 1 else 0


def _pollard(n, rounds=24):
    from math import gcd
    for k in range(1, rounds):
```

```
        x = y = 2
        d = 1
        while d == 1:
            x = (x * x + k) % n
            y = (y * y + k) % n
            y = (y * y + k) % n
            d = gcd(abs(x - y), n)
        if d != n:
            return d
    return None


def mod8_reject(p, a, c):
    """True when sigma(p^(2a)) is not 1 or 3 mod 8, which for c = 1 rules out
    p^(2a) as an exact component. Free: no factoring, no large arithmetic.

    By Lemma qr every prime factor of sigma(n) is 1 or 3 mod 8, and {1,3} is
    closed under multiplication mod 8, so sigma(p^(2a)) -- a product of such
    primes -- is itself 1 or 3 mod 8. Since p^2 = 1 mod 8, the 2a+1 terms of
    sigma(p^(2a)) comprise a+1 even powers (each 1) and a odd powers (each p), so
        sigma(p^(2a)) = 1 + a(p+1)   (mod 8).
    Verified over 2,256 (p, a) pairs. The surviving exponents are a = 0 or 1 mod 4
    for p = 1 mod 8, a even for p = 3 mod 8, a = 0 or 3 mod 4 for p = 5 mod 8, and
    anything for p = 7 mod 8 -- three of the four conditions of Cattaneo's 1951
    theorem, derived here rather than taken from it.

    Guarded to c = 1: for other c the admissible residues of a prime factor are
    those with (-2c/r) = 1, which need not form a multiplicative subgroup, so the
    step from "each factor" to "the product" fails.
    """
    if c != 1:
        return False
    return (1 + a * (p + 1)) % 8 not in (1, 3)


_QR_CACHE = {}


def qr_reject(p, a, c, tdiv=1 << 12, digits=25):
    """True only when sigma(p^(2a)) provably has a prime factor r, coprime to 2c,
    with (-2c/r) != 1 -- in which case p^(2a) cannot be the exact power of p in a
    solution of sigma(n) = 2n + c with n an odd square.

    Reason: r | sigma(p^(2a)) | sigma(n) = 2n + c and n = m^2 give
    (2m)^2 = -2c (mod r), so -2c must be a quadratic residue mod r.

    One-sided by design. Factoring is attempted only so far as is cheap, so a
    return of False means "not shown to be impossible", never "possible". That
    keeps the prune sound however little factoring succeeds.
    """
    key = (p, a, c)
    hit = _QR_CACHE.get(key)
    if hit is not None:
        return hit

    def bad(r):
        return c % r and jacobi(-2 * c % r, r) != 1

    s = sig(p, 2 * a)
    verdict = False
    # small primes first: cheap, and catches most cases
    for d in range(3, tdiv, 2):
        if d * d > s:
            break
        if s % d == 0:
            if bad(d):
                verdict = True
                break
            while s % d == 0:
                s //= d
    if not verdict and s > 1 and not is_probable_prime(s):
        # one cheap rho attempt, but not on numbers too big to bother with
        if len(str(s)) <= digits:
            stack = [s]
            while stack and not verdict:
                t = stack.pop()
```

```
                if t == 1:
                    continue
                if is_probable_prime(t):
                    if bad(t):
                        verdict = True
                    continue
                d = _pollard(t)
                if d is None:
                    break
                stack += [d, t // d]
    elif not verdict and s > 1:
        verdict = bad(s)

    _QR_CACHE[key] = verdict
    return verdict


def isqrt_ceil_div(a, b):
    return -(-a // b)


class Search:
    def __init__(self, c=1, q_limit=10**7, log=None, refine_width=64, refine_qmax=5000,
                 use_qr=True, exclude=()):
        self.c = c
        self.use_qr = use_qr
        # Primes barred from dividing n. Used for conditional bounds: with
        # exclude = {3, 5} the search settles omega for quasiperfect numbers
        # coprime to 15, where the abundancy ceiling is far enough from 2 that
        # many more primes are needed and the tree is correspondingly narrow.
        self.exclude = frozenset(exclude)
        self.refine_width = refine_width
        self.refine_qmax = refine_qmax
        self.q_limit = q_limit   # max primes enumerated at one node before flagging
        self.nodes = 0
        self.leaf_checks = 0
        self.solutions = []
        self.unresolved = []
        self.log = log

    # comps: tuple of (p, a, fixed); exponent 2a, or >= 2a when not fixed.
    def bounds(self, comps):
        lo = F(1)
        hi = F(1)
        nmin = 1
        for p, a, fx in comps:
            pe = p ** (2 * a)
            h = F(sig(p, 2 * a), pe)
            lo *= h
            nmin *= pe
            hi *= h if fx else F(p, p - 1)
        return lo, hi, nmin

    # (4): a prime of n dividing sigma(p^(2a)) must divide c
    # (8): every prime of sigma(p^(2a)) must have -2c as a quadratic residue
    def coprime_ok_fix(self, comps, p, a):
        s = sig(p, 2 * a)
        if not all(s % q or self.c % q == 0 for q, _, _ in comps if q != p):
            return False
        if not self.use_qr:
            return True
        # free test first, then the one that has to factor
        if mod8_reject(p, a, self.c):
            return False
        return not qr_reject(p, a, self.c)

    def coprime_ok_add(self, comps, q):
        if self.c % q == 0:
            return True
        return all(sig(p, 2 * a) % q for p, a, fx in comps if fx)

    def record(self, n):
        # callers have already checked sigma(n) = 2n + c exactly from the factorisation
        self.solutions.append(n)

    def run(self, k):
```

```
        self.search((), k)

    def search(self, comps, r):
        self.nodes += 1
        self.current = comps
        if self.log and self.nodes % 200_000 == 0:
            self.log(self)
        c = self.c
        lo, hi, nmin = self.bounds(comps)
        last = comps[-1][0] if comps else 2
        opens = [i for i, t in enumerate(comps) if not t[2]]

        if r == 0:
            if not opens:
                self.leaf_checks += 1
                if lo * nmin == 2 * nmin + c:
                    self.record(nmin)
                return
            if hi <= 2:                       # (1): open parts stay strictly below hi
                return
            if lo > 2 + F(c, nmin):           # (1)
                return
            if len(opens) == 1:
                self.single_open(comps, opens[0])
                return
            self.refine(comps, opens, r)
            return

        # r >= 1
        if lo >= 2:
            # (1)+(2): lo(1 + 1/q) < 2 + c/(nmin q^2)  =>  q < c/(nmin lo)
            qb = F(c) / (nmin * lo)
            q = next_prime(last)
            while q < qb:                     # never entered when c = 1
                if q not in self.exclude and self.coprime_ok_add(comps, q):
                    self.search(comps + ((q, 1, False),), r - 1)
                q = next_prime(q)
            return
        # (3)
        ub = hi
        q = last
        for _ in range(r):
            q = next_prime(q)
            ub *= F(q, q - 1)
        if ub <= 2:
            return
        if hi >= 2:
            self.refine(comps, opens, r)
            return
        if r == 1 and not opens:
            self.last_prime(comps, lo, nmin, last)
            return
        if r >= 2 and opens:
            # When hi is very close to 2 the next prime ranges over a huge
            # interval, and every choice would re-refine the same open exponents.
            # Refine them once, here, instead: most exponent choices pull hi well
            # below 2 and the branch dies at once. hi alone does not shrink as
            # exponents grow, so the test also uses the lower end q_lo, which
            # rises towards q_hi / r; the condition therefore fails eventually
            # and refinement terminates. Ordering only, not a prune.
            q_hi = r * hi / (2 - hi)
            q_lo = lo / (2 - lo + F(c, nmin))
            if q_hi > self.refine_qmax and q_hi > 4 * r * q_lo:
                self.refine(comps, opens, r)
                return
        if r == 1:
            # The last prime lies in (lo/(2-lo), 2/(2-hi)), a range whose width
            # tends to 1 as the open intervals tighten. Tighten them first
            # rather than enumerate a wide range with every exponent still open.
            # Ordering only, not a prune. (Doing the same at r >= 2 was tried
            # and made omega = 5 ten times slower.)
            width = F(2) / (2 - hi) - lo / (2 - lo + F(c, nmin))
            if width > self.refine_width:
                self.refine(comps, opens, r)
                return
        self.enumerate_next(comps, r, lo, hi, nmin, last)
```

```
321
322     def refine(self, comps, opens, r):
323         # split the open component with the smallest exponent floor (round robin)
324         i = min(opens, key=lambda j: (comps[j][1], comps[j][0]))
325         p, a, _ = comps[i]
326         if self.coprime_ok_fix(comps, p, a):
327             self.search(comps[:i] + ((p, a, True),) + comps[i + 1:], r)
328         self.search(comps[:i] + ((p, a + 1, False),) + comps[i + 1:], r)
329
330     def single_open(self, comps, i):
331         # (5)
332         c = self.c
333         p, A, _ = comps[i]
334         s_rest = 1
335         n_rest = 1
336         for j, (q, a, _) in enumerate(comps):
337             if j != i:
338                 s_rest *= sig(q, 2 * a)
339                 n_rest *= q ** (2 * a)
340         T = s_rest + c * (p - 1)
341         v = 0
342         while T % p == 0:
343             T //= p
344             v += 1
345         for a in range(A, v // 2 + 1):
346             self.leaf_checks += 1
347             pe = p ** (2 * a)
348             if s_rest * sig(p, 2 * a) == 2 * n_rest * pe + c:
349                 self.record(n_rest * pe)
350
351     def last_prime(self, comps, lo, m, last):
352         # (6): q^(2a) (2m - Dq) = c q + S - c, with q > last and q < 2m/D.
353         c = self.c
354         S = lo.numerator * (m // lo.denominator)
355         assert F(S, m) == lo
356         D = 2 * m - S
357         assert D > 0
358         qmax = (2 * m - 1) // D               # 2m - Dq >= 1
359         qmin = last + 1
360         if qmax < qmin:
361             return
362         a = 1
363         while True:
364             e = 2 * a
365             if qmin ** e > c * qmax + S:      # g(q) >= q^e > RHS for every q
366                 break
367
368             for q in poly_roots(m, S, D, c, e, qmin, qmax):
369                 self.leaf_checks += 1
370                 if S * sig(q, e) == 2 * m * q ** e + c:
371                     self.record(m * q ** e)
372             a += 1
373
374     def enumerate_next(self, comps, r, lo, hi, nmin, last):
375         c = self.c
376         # (2): lo (1 + 1/q) < 2 + c/nmin  =>  q > lo / (2 - lo + c/nmin)
377         bound = lo / (2 - lo + F(c, nmin))
378         start = max(last, bound.numerator // bound.denominator - 1)
379         q = next_prime(start)
380         count = 0
381         while True:
382             # (3) with q as the smallest remaining prime
383             ub = hi
384             t = q
385             ub *= F(t, t - 1)
386             for _ in range(r - 1):
387                 t = next_prime(t)
388                 ub *= F(t, t - 1)
389             if ub <= 2:
390                 break
391             if (q not in self.exclude
392                     and lo * F(q * q + q + 1, q * q) <= 2 + F(c, nmin * q * q)
393                     and self.coprime_ok_add(comps, q)):
394                 self.search(comps + ((q, 1, False),), r - 1)
395             count += 1
396             if count > self.q_limit:
```

```
                self.unresolved.append(("q-range", comps, r, q))
                return
            q = next_prime(q)


def poly_roots(m, S, D, c, e, lo, hi):
    """All integers q in [lo, hi] with P(q) = q^e (2m - Dq) - cq - (S - c) = 0.

    P'' = e q^(e-2) ((e-1) 2m - (e+1) D q) changes sign once, at q_inf, so P' rises
    then falls and P has at most three monotone stretches. Find P' = 0 on each
    side of q_inf by bisection, then bisect P on each stretch.
    """
    def P(q):
        return q ** e * (2 * m - D * q) - c * q - (S - c)

    def dP(q):
        return e * 2 * m * q ** (e - 1) - (e + 1) * D * q ** e - c

    if lo > hi:
        return []
    q_inf = ((e - 1) * 2 * m) // ((e + 1) * D)

    def first_true(a, b, pred):          # smallest x in [a, b] with pred, else b + 1
        while a < b:
            mid = (a + b) // 2
            if pred(mid):
                b = mid
            else:
                a = mid + 1
        return a if pred(a) else a + 1

    far = max(hi, q_inf + 1) + 1
    x1 = first_true(0, q_inf, lambda x: dP(x) >= 0)           # P' rising here
    x2 = first_true(q_inf + 1, far, lambda x: dP(x) < 0)      # P' falling here
    cuts = {lo, hi, q_inf, q_inf + 1, x1 - 1, x1, x2 - 1, x2}
    pts = sorted(x for x in cuts if lo <= x <= hi)
    roots = set()
    for x in pts:
        if P(x) == 0:
            roots.add(x)
    for x, y in zip(pts, pts[1:]):
        px, py = P(x), P(y)
        if px == 0 or py == 0 or (px > 0) == (py > 0):
            continue
        up = py > px
        a, b = x, y
        while b - a > 1:
            mid = (a + b) // 2
            pm = P(mid)
            if pm == 0:
                roots.add(mid)
                break
            if (pm < 0) == up:
                a = mid
            else:
                b = mid
    return sorted(roots)


def factorize(n):
    f = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            f[d] = f.get(d, 0) + 1
            n //= d
        d += 1 if d == 2 else 2
    if n > 1:
        f[n] = f.get(n, 0) + 1
    return f


def sigma_of(n):
    s = 1
    for p, e in factorize(n).items():
        s *= sig(p, e)
```

```
    return s


if __name__ == "__main__":
    kmax = int(sys.argv[1]) if len(sys.argv) > 1 else 6
    c = int(sys.argv[2]) if len(sys.argv) > 2 else 1
    for k in range(1, kmax + 1):
        t0 = time.time()
        s = Search(c=c, log=lambda s: print(f"  ... {s.nodes:,} nodes", flush=True))
        s.run(k)
        print(f"c={c} omega={k}: nodes={s.nodes:,} leaf checks={s.leaf_checks:,} "
              f"solutions={s.solutions} unresolved={len(s.unresolved)} "
              f"({time.time() - t0:.1f}s)", flush=True)
        for u in s.unresolved[:5]:
            print("   unresolved:", u)
```

1.2 run_parallel.py

Cuts the tree into independent subtrees and runs them.

```
"""Split the omega = k search into independent subtrees and run them in parallel.

The tree is walked normally until a node has `depth` primes; that node becomes
a task instead of being expanded. Every task is an ordinary Search.search call,
so the union of tasks is exactly the full tree. Finished tasks are appended to
results/omega<k>_d<depth>.jsonl, so an interrupted run resumes where it left off.

Usage: python run_parallel.py K [DEPTH] [WORKERS]
"""
import json
import os
import sys
import time
from multiprocessing import Pool

from priority import raise_priority
from qp import Search


class Splitter(Search):
    def __init__(self, depth, exclude=()):
        super().__init__(c=1, exclude=exclude)
        self.depth = depth
        self.tasks = []

    def search(self, comps, r):
        if len(comps) == self.depth and r > 0:
            self.tasks.append((comps, r))
            return
        super().search(comps, r)


def key(task):
    return json.dumps([[list(c) for c in task[0]], task[1]])


HERE = os.path.dirname(os.path.abspath(__file__))
PAUSE_FILE = os.path.join(HERE, "results", "PAUSE")
PROGRESS_DIR = os.path.join(HERE, "results", "progress")


def _write_progress(path, rec):
    """Replace a worker's progress file atomically, so a reader never sees half."""
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        json.dump(rec, f)
    os.replace(tmp, path)


def _pause_hook(paused, task, t0, c0):
    """Search.log callback, called every 200,000 states.

    Does two jobs through the filesystem, the one channel shared with runs the
    S4U scheduled task launches in session 0 - where an ordinary session cannot
    open the processes at all, not to query, suspend or kill:

```

```
    - waits while results/PAUSE exists;
    - writes results/progress/<pid>.json with the subtree and the states searched
      so far. Without this a running subtree is a black box until it finishes:
      on 4 Oct 2026 nine omega = 8 workers had run 13.5 hours each with nothing
      to say whether they were 10% done or 0.1%.

    Costs one small file write per 200,000 states, which is noise.
    """
    os.makedirs(PROGRESS_DIR, exist_ok=True)
    path = os.path.join(PROGRESS_DIR, f"{os.getpid()}.json")

    def hook(search):
        wall = time.time() - t0 - paused[0]
        _write_progress(path, {
            "task": key(task), "pid": os.getpid(), "nodes": search.nodes,
            "seconds": round(wall, 1),
            "cpu_seconds": round(time.process_time() - c0, 1),
            "paused_seconds": round(paused[0], 1),
            "paused": os.path.exists(PAUSE_FILE), "updated": time.time()})
        if os.path.exists(PAUSE_FILE):
            p0 = time.time()
            while os.path.exists(PAUSE_FILE):
                time.sleep(5)
            paused[0] += time.time() - p0
    hook.path = path
    return hook


def run_task(task):
    # Every worker opts out of Windows' background power throttling. Without
    # this the pool runs at about a quarter speed; see priority.py.
    prio = raise_priority()
    # (comps, r) or (comps, r, exclude): the barred primes must reach the worker,
    # or a parallel conditional search would silently become unconditional.
    comps, r = task[0], task[1]
    excl = task[2] if len(task) > 2 else ()
    paused = [0.0]
    t0, c0 = time.time(), time.process_time()
    hook = _pause_hook(paused, task, t0, c0)
    s = Search(c=1, exclude=excl, log=hook)
    s.search(tuple(tuple(c) for c in comps), r)
    wall = time.time() - t0 - paused[0]
    cpu = time.process_time() - c0
    try:                                    # finished: its result goes in the jsonl
        os.remove(hook.path)
    except OSError:
        pass
    # cpu/wall is the throttling gauge: ~1.0 means this worker had a whole core,
    # ~0.3 means Windows was holding it back. Recorded per subtree, so it is
    # visible even for workers this session cannot open.
    return {"task": key(task), "nodes": s.nodes, "leaf_checks": s.leaf_checks,
            "solutions": [str(n) for n in s.solutions],
            "unresolved": [repr(u) for u in s.unresolved],
            "seconds": round(wall, 2), "cpu_seconds": round(cpu, 2),
            "paused_seconds": round(paused[0], 1),
            "priority_ok": prio["priority"], "ecoqos_off": prio["ecoqos_off"]}


def main():
    k = int(sys.argv[1])
    depth = int(sys.argv[2]) if len(sys.argv) > 2 else 3
    workers = int(sys.argv[3]) if len(sys.argv) > 3 else max(1, os.cpu_count() - 2)

    sp = Splitter(depth)
    sp.run(k)
    head = {"nodes": sp.nodes, "solutions": sp.solutions, "unresolved": sp.unresolved}
    tasks = sp.tasks

    os.makedirs("results", exist_ok=True)
    path = f"results/omega{k}_d{depth}.jsonl"
    done = {}
    if os.path.exists(path):
        with open(path) as f:
            for line in f:
                rec = json.loads(line)
                done[rec["task"]] = rec
```

```
    todo = [t for t in tasks if key(t) not in done]
    print(f"omega={k} depth={depth}: {len(tasks)} tasks, {len(done)} already done, "
          f"{len(todo)} to run on {workers} workers; head tree {head}", flush=True)

    t0 = time.time()
    with open(path, "a") as out, Pool(workers) as pool:
        for i, rec in enumerate(pool.imap_unordered(run_task, todo, chunksize=1), 1):
            out.write(json.dumps(rec) + "\n")
            out.flush()
            done[rec["task"]] = rec
            if rec["solutions"] or rec["unresolved"]:
                print("FLAG", rec, flush=True)
            if i % 50 == 0 or i == len(todo):
                print(f"  {len(done)}/{len(tasks)} tasks, {time.time() - t0:.0f}s", flush=True)

    recs = [done[key(t)] for t in tasks]
    nodes = head["nodes"] + sum(r["nodes"] for r in recs)
    sols = head["solutions"] + [s for r in recs for s in r["solutions"]]
    unres = len(head["unresolved"]) + sum(len(r["unresolved"]) for r in recs)
    print(f"DONE omega={k}: nodes={nodes:,} solutions={sols} unresolved={unres}", flush=True)


if __name__ == "__main__":
    main()
```

1.3 split_task.py

Splits one subtree further, for the case of Issue 7.

```
"""Split one task from a run_parallel results file one level deeper and run it in parallel.

Usage: python split_task.py K DEPTH TASK_JSON [WORKERS]

The sub-tasks come from walking TASK's subtree with Splitter(DEPTH), so together
they cover exactly TASK. Finished sub-tasks go to results/split_<hash>.jsonl
(resumable). When all are done, one record for TASK, marked "split", is appended
to results/omega<K>_d<depth of TASK>.jsonl so the main run treats it as done.
"""
import hashlib
import json
import os
import sys
import time
from multiprocessing import Pool

from run_parallel import Splitter, key, run_task


def main():
    k, depth, task_json = int(sys.argv[1]), int(sys.argv[2]), sys.argv[3]
    workers = int(sys.argv[4]) if len(sys.argv) > 4 else 13
    comps, r = json.loads(task_json)
    comps = tuple(tuple(c) for c in comps)
    task = (comps, r)

    sp = Splitter(depth)
    sp.search(comps, r)            # walks the subtree, collecting sub-tasks
    head_nodes = sp.nodes
    subs = sp.tasks

    os.makedirs("results", exist_ok=True)
    tag = hashlib.sha1(key(task).encode()).hexdigest()[:10]
    path = f"results/split_{tag}.jsonl"
    done = {}
    if os.path.exists(path):
        for line in open(path):
            rec = json.loads(line)
            done[rec["task"]] = rec
    todo = [t for t in subs if key(t) not in done]
    print(f"{key(task)} -> {len(subs)} sub-tasks at depth {depth}, "
          f"{len(done)} done, head nodes {head_nodes}, unresolved {sp.unresolved}", flush=True)

    t0 = time.time()
    with open(path, "a") as out, Pool(workers) as pool:
        for i, rec in enumerate(pool.imap_unordered(run_task, todo, chunksize=1), 1):
            out.write(json.dumps(rec) + "\n")
```

```
            out.flush()
            done[rec["task"]] = rec
            if rec["solutions"] or rec["unresolved"]:
                print("FLAG", rec, flush=True)
            if i % 25 == 0 or i == len(todo):
                print(f"  {len(done)}/{len(subs)} sub-tasks, {time.time() - t0:.0f}s", flush=True)

    recs = [done[key(t)] for t in subs]
    merged = {
        "task": key(task),
        "nodes": head_nodes + sum(x["nodes"] for x in recs),
        "leaf_checks": sp.leaf_checks + sum(x["leaf_checks"] for x in recs),
        "solutions": [str(n) for n in sp.solutions] + [s for x in recs for s in x["solutions"]],
        "unresolved": [repr(u) for u in sp.unresolved] + [u for x in recs for u in x["unresolved"]],
        "seconds": round(sum(x["seconds"] for x in recs), 2),
        "split": {"depth": depth, "subtasks": len(subs), "file": path},
    }
    main_path = f"results/omega{k}_d{len(comps)}.jsonl"
    existing = {json.loads(l)["task"] for l in open(main_path)} if os.path.exists(main_path) else set()
    if merged["task"] not in existing:
        with open(main_path, "a") as f:
            f.write(json.dumps(merged) + "\n")
    print("MERGED", {k2: v for k2, v in merged.items() if k2 != "task"}, flush=True)


if __name__ == "__main__":
    main()
```

1.4 audit_results.py

Audits a finished run for complete coverage.

```
"""Audit a completed run: does the recorded evidence actually cover the whole tree?

The claim "no solution with this many prime factors" rests on the union of the
recorded subtrees being the entire search tree. That is checkable without
re-running anything: re-derive the task list from the splitter, then require
every task to have a record, no record to be for an unknown task, no task to
be left unresolved, and no solution to have been found. Duplicate records for
the same task are reported and reconciled rather than silently summed.

Usage: python audit_results.py K DEPTH
"""
import json
import os
import sys
from collections import defaultdict

from run_parallel import Splitter, key


def main():
    k, depth = int(sys.argv[1]), int(sys.argv[2])
    sp = Splitter(depth)
    sp.run(k)
    expected = {key(t) for t in sp.tasks}

    path = f"results/omega{k}_d{depth}.jsonl"
    recs = defaultdict(list)
    for line in open(path):
        r = json.loads(line)
        recs[r["task"]].append(r)

    missing = expected - recs.keys()
    unknown = recs.keys() - expected
    dupes = {t: v for t, v in recs.items() if len(v) > 1}

    print(f"tasks expected from the splitter : {len(expected)}")
    print(f"tasks with at least one record   : {len(recs)}")
    print(f"records in file                  : {sum(len(v) for v in recs.values())}")
    print(f"missing tasks                    : {len(missing)}")
    print(f"records for unexpected tasks     : {len(unknown)}")
    print(f"tasks with duplicate records     : {len(dupes)}")
    for t, v in dupes.items():
        print(f"  duplicate: {t}")
        for r in v:
```

```
            tag = f"split into {r['split']['subtasks']}" if "split" in r else "single process"
            print(f"    nodes={r['nodes']:>13,}  secs={r['seconds']:>10,.0f}  {tag}")

    # One record per task for the totals: prefer the first, and check the
    # duplicates agree on the only thing that matters (nothing found, nothing left).
    chosen = {t: v[0] for t, v in recs.items()}
    nodes = sp.nodes + sum(r["nodes"] for r in chosen.values())
    leaves = sp.leaf_checks + sum(r["leaf_checks"] for r in chosen.values())
    sols = [s for r in recs.values() for x in r for s in x["solutions"]] + \
           [str(n) for n in sp.solutions]
    unres = [u for r in recs.values() for x in r for u in x["unresolved"]] + \
            [repr(u) for u in sp.unresolved]

    print(f"\nhead tree (above the split)      : {sp.nodes:,} nodes")
    print(f"total nodes (one record per task): {nodes:,}")
    print(f"exact leaf checks                : {leaves:,}")
    print(f"solutions, across ALL records    : {sols}")
    print(f"unresolved, across ALL records   : {len(unres)}")

    ok = not missing and not unknown and not sols and not unres
    print(f"\n{'COMPLETE: the recorded subtrees cover the whole tree, nothing found, nothing unresolved.' if ok else
    ↪  'INCOMPLETE - see above.'}")
    return 0 if ok else 1


if __name__ == "__main__":
    sys.exit(main())
```

1.5 test_qp.py

Cross-check against brute force over odd squares.

```
"""Cross-check the tree search against brute force on sigma(n) = 2n + c.

For every odd k <= K, n = k^2 is checked directly. For each c > 0 that
brute force hits, the solver must find every brute-force solution, and every
solver solution <= K^2 must appear in the brute-force list.
"""
import sys
import time
from collections import defaultdict

from qp import Search, factorize, sig

K = int(sys.argv[1]) if len(sys.argv) > 1 else 20001
CMAX = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

brute = defaultdict(list)          # (c, omega) -> [n]
for k in range(3, K + 1, 2):
    f = factorize(k)
    n = k * k
    s = 1
    for p, e in f.items():
        s *= sig(p, 2 * e)
    c = s - 2 * n
    if 0 < c <= CMAX:
        brute[(c, len(f))].append(n)

print(f"brute force: {sum(map(len, brute.values()))} hits over {len(brute)} (c, omega) pairs")
bad = 0
t0 = time.time()
for (c, w), ns in sorted(brute.items()):
    s = Search(c=c)
    s.run(w)
    found = sorted(s.solutions)
    small = [n for n in found if n <= K * K]
    ok = sorted(ns) == small and not s.unresolved
    if not ok:
        bad += 1
    print(f"c={c:5d} omega={w}: brute={sorted(ns)} solver={found} "
          f"unresolved={len(s.unresolved)} nodes={s.nodes} {'OK' if ok else 'MISMATCH'}")
print(f"{bad} mismatches, {time.time() - t0:.1f}s")
```

1.6 test_leaves.py

Planted solutions for the two closing lemmas.

```
"""Unit tests for the two leaf solvers on planted solutions of sigma(n) = 2n + c.

Brute force cannot reach large exponents or large last primes, so plant n from
a chosen factorisation, set c = sigma(n) - 2n, and check that each leaf solver
recovers the missing piece from the rest:

  last_prime:  every other component fixed, the largest prime unknown.
  single_open: every prime known, one exponent open from 2.
"""
import random
import sys

from qp import Search, is_probable_prime, sig

rng = random.Random(int(sys.argv[2]) if len(sys.argv) > 2 else 1)
N = int(sys.argv[1]) if len(sys.argv) > 1 else 2000
odd_primes = [p for p in range(3, 2000) if is_probable_prime(p)]

done = {"last_prime": 0, "single_open": 0}
fails = 0
while min(done.values()) < N:
    w = rng.randint(2, 5)
    ps = sorted(rng.sample(odd_primes[:15], w - 1))
    big = rng.choice([p for p in odd_primes if p > ps[-1]])
    ps.append(big)
    exps = [rng.choice([1, 1, 1, 2, 3, 4, 7, 12]) for _ in ps]
    n, s = 1, 1
    for p, a in zip(ps, exps):
        n *= p ** (2 * a)
        s *= sig(p, 2 * a)
    c = s - 2 * n
    if c <= 0:
        continue
    fixed = tuple((p, a, True) for p, a in zip(ps, exps))

    # last prime: the rest must be deficient (h < 2) for the leaf to apply
    m, sm = 1, 1
    for p, a in zip(ps[:-1], exps[:-1]):
        m *= p ** (2 * a)
        sm *= sig(p, 2 * a)
    if sm < 2 * m and done["last_prime"] < N:
        S = Search(c=c)
        S.last_prime(fixed[:-1], __import__("fractions").Fraction(sm, m), m, ps[-2])
        done["last_prime"] += 1
        if n not in S.solutions or S.unresolved:
            fails += 1
            print("last_prime MISS", ps, exps, c, S.solutions, S.unresolved[:1])

    # single open exponent, any position
    if done["single_open"] < N:
        i = rng.randrange(w)
        comps = fixed[:i] + ((ps[i], 1, False),) + fixed[i + 1:]
        S = Search(c=c)
        S.single_open(comps, i)
        done["single_open"] += 1
        if n not in S.solutions:
            fails += 1
            print("single_open MISS", ps, exps, i, c, S.solutions)

print(f"{done} planted, {fails} failures")
```

1.7 peek.py

Samples which states the search is visiting.

```
"""Print where the search is every N nodes, to see what dominates the tree."""
import sys

from qp import Search

k = int(sys.argv[1])
every = int(sys.argv[2]) if len(sys.argv) > 2 else 20000
```

```
stop = int(sys.argv[3]) if len(sys.argv) > 3 else 400000


class Stop(Exception):
    pass


def log(s):
    print(s.nodes, s.current, flush=True)
    if s.nodes >= stop:
        raise Stop


s = Search(c=1, log=log)
import qp
qp_log_every = every
orig = Search.search


def search(self, comps, r):
    if self.nodes % every == 0 and self.nodes:
        print(self.nodes, r, comps, flush=True)
        if self.nodes >= stop:
            raise Stop
    return orig(self, comps, r)


Search.search = search
try:
    s.run(k)
except Stop:
    pass
```

1.8 qfast.py

Compiled search that closed the last omega = 17 node.

```
"""Compiled search of one r = 2 node whose next-prime range is too long for Search.

Built for the single node omega = 17 (3 and 5 barred) left unresolved
("q-range", stopped by q_limit at q = 571,287,683): fifteen components, nine
with open exponents, two primes q < q' still to choose. Search spends ~8,000
states per q there, and the remaining range holds ~27 million primes.

Method. Write the abundancy in logarithms. With hi the node's upper bound (open
exponents at infinity) and K = log(2/hi) > 0, an open component p with exponent
2a contributes log(p/(p-1)) - d_p(a), where d_p(a) = -log(1 - p^-(2a+1)), so

    log f(q) + log f(q') = K + sum_p d_p + log(1 + 1/(2N)),

with f(x) = sigma(x^2b)/x^2b in [log1p((x+1)/x^2), -log1p(-1/x)) for every b >= 1.
For each q the search keeps a box over the open exponents (each either fixed at
a or open at >= a, contributing [d_p(a), d_p(a)] or [0, d_p(a)]), turns the box
into the range of q' it allows, splits the open component with the widest
contribution until that range is short, then tries each prime q' in it and
splits on until the box is narrower than the target the pair (q, q') leaves.

Sums are in double-double arithmetic (~32 significant digits; the big terms
are ~1.8e-9, so absolute error ~1e-40), every table value is rounded from an
exact rational, and every comparison carries a slack S = 1e-35 far above the
rounding error, so the search can only keep too much, never prune a real
solution. (A first plain-double version needed S = 2e-23 and left ~0.56 near
misses per q for the exact code, ~75 ms each.) Whatever survives to the bottom is written
out as a candidate and settled by the original exact code (Search.search from
that box with q and q' as open components). The term log(1 + 1/(2N)) is below
1e-140 and inside S.

Prunes of Search not used here (coprimality with q and q', the residue test on
q and q') only shrink its tree, so dropping them is sound. The residue/mod-8
test on fixing an open exponent is kept, precomputed per (p, a) with
coprime_ok_fix against the node's own primes.
"""
import math
import numpy as np
from numba import njit

```

```
S_SLACK = 1e-35
W_MAX = 32
D_FLOOR = 1e-50          # exponents whose d_p(a) is below this are left open


@njit(cache=True)
def mulmod(a, b, n):                       # n < 2^38
    return (((a * (b >> 19)) % n << 19) + a * (b & 524287)) % n


@njit(cache=True)
def powmod(a, e, n):
    r = 1
    a %= n
    while e:
        if e & 1:
            r = mulmod(r, a, n)
        a = mulmod(a, a, n)
        e >>= 1
    return r


@njit(cache=True)
def isprime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in (2, 3, 5, 7, 11, 13, 17):         # deterministic below 3.4e14
        x = powmod(a, d, n)
        if x == 1 or x == n - 1:
            continue
        comp = True
        for _ in range(s - 1):
            x = mulmod(x, x, n)
            if x == n - 1:
                comp = False
                break
        if comp:
            return False
    return True


@njit(cache=True)
def dd_add(ah, al, bh, bl):
    s = ah + bh
    v = s - ah
    e = (ah - (s - v)) + (bh - v) + al + bl
    h = s + e
    return h, e - (h - s)


@njit(cache=True)
def two_prod(a, b):
    p = a * b
    # Dekker split
    c = 134217729.0 * a
    ah = c - (c - a)
    al = a - ah
    c = 134217729.0 * b
    bh = c - (c - b)
    bl = b - bh
    return p, ((ah * bh - p) + ah * bl + al * bh) + al * bl


@njit(cache=True)
def dd_recip(x):
    # 1/x for an integer x < 2^53, as a double-double
    h = 1.0 / x
    p, pe = two_prod(h, x)          # h x = 1 + r
```

```
    r = (p - 1.0) + pe
    return h, -h * r


@njit(cache=True)
def log_f_bounds(x):
    # (log(1 + 1/x + 1/x^2), -log(1 - 1/x)) as double-doubles, x ~ 1e8..1e11
    yh, yl = dd_recip(float(x))
    # y^2 and u^2 are ~3e-18, so they need their own low parts; cubes and up
    # are below 1e-26 and a double is ample.
    s2, s2l = two_prod(yh, yh)
    s2l += 2.0 * yh * yl
    y3 = s2 * yh
    y4 = s2 * s2
    # u = y + y^2; log1p(u) = u - u^2/2 + u^3/3 - u^4/4
    uh, ul = dd_add(yh, yl, s2, s2l)
    u2, u2l = two_prod(uh, uh)
    u2l += 2.0 * uh * ul
    u3 = u2 * uh
    u4 = u2 * u2
    loh, lol = dd_add(uh, ul, -0.5 * u2, -0.5 * u2l)
    loh, lol = dd_add(loh, lol, u3 / 3.0 - 0.25 * u4, 0.0)
    # -log1p(-y) = y + y^2/2 + y^3/3 + y^4/4
    hih, hil = dd_add(yh, yl, 0.5 * s2, 0.5 * s2l)
    hih, hil = dd_add(hih, hil, y3 / 3.0 + 0.25 * y4, 0.0)
    return loh, lol, hih, hil


@njit(cache=True)
def run_q(q, K, KL, dtab, dtl, allowed, floor, amax, out, nout, stats):
    """Search one q. Candidates appended to out[nout:] as (q, q', a_0..a_k, fx_0..fx_k)."""
    nop = floor.shape[0]
    maxd = 4096
    st_a = np.empty((maxd, nop), np.int64)
    st_f = np.empty((maxd, nop), np.int64)
    st_p = np.empty(maxd, np.int64)
    top = 0
    for i in range(nop):
        st_a[0, i] = floor[i]
        st_f[0, i] = 0
    st_p[0] = 0
    top = 1
    qlo, qlol, qhi, qhil = log_f_bounds(q)
    a = np.empty(nop, np.int64)
    fx = np.empty(nop, np.int64)
    while top > 0:
        top -= 1
        stats[0] += 1
        for i in range(nop):
            a[i] = st_a[top, i]
            fx[i] = st_f[top, i]
        qp = st_p[top]
        Dlo = 0.0
        Dll = 0.0
        Dhi = 0.0
        Dhl = 0.0
        best = -1
        bestd = 0.0
        for i in range(nop):
            if a[i] > amax[i]:
                Dhi, Dhl = dd_add(Dhi, Dhl, D_FLOOR, 0.0)
                continue
            d = dtab[i, a[i]]
            dl = dtl[i, a[i]]
            if fx[i]:
                Dlo, Dll = dd_add(Dlo, Dll, d, dl)
                Dhi, Dhl = dd_add(Dhi, Dhl, d, dl)
            else:
                Dhi, Dhl = dd_add(Dhi, Dhl, d, dl)
                if d > bestd:
                    bestd = d
                    best = i
        if qp == 0:
            h, l = dd_add(K, KL, Dhi, Dhl)
            h, l = dd_add(h, l, -qlo, -qlol)
            Rhi = h + l + S_SLACK
```

```
            h, l = dd_add(K, KL, Dlo, Dll)
            h, l = dd_add(h, l, -qhi, -qhil)
            Rlo = h + l - S_SLACK
            if Rhi <= 0.0:
                continue
            y = math.expm1(Rhi)
            Lb = (1.0 + math.sqrt(1.0 + 4.0 * y)) / (2.0 * y)
            lo_i = max(int(math.floor(Lb)) - 1, q + 1)
            if Rlo <= 0.0:
                hi_i = 1 << 40
            else:
                hi_i = int(math.ceil(1.0 / (-math.expm1(-Rlo)))) + 1
            if hi_i < lo_i:
                continue
            if hi_i - lo_i > W_MAX and best >= 0:
                pass                                    # split below
            else:
                if hi_i - lo_i > 10_000_000 or hi_i >= (1 << 38):
                    stats[2] += 1                       # unresolved
                    continue
                for qq in range(lo_i, hi_i + 1):
                    if qq % 2 == 1 and isprime(qq):
                        if top >= maxd:
                            stats[3] += 1
                            return nout
                        for i in range(nop):
                            st_a[top, i] = a[i]
                            st_f[top, i] = fx[i]
                        st_p[top] = qq
                        top += 1
                continue
        else:
            plo, plol, phi, phil = log_f_bounds(qp)
            # need Dhi >= qlo + plo - K and Dlo <= qhi + phi - K (within S)
            h, l = dd_add(Dhi, Dhl, K, KL)
            h, l = dd_add(h, l, -qlo, -qlol)
            h, l = dd_add(h, l, -plo, -plol)
            if h + l < -S_SLACK:
                continue
            h, l = dd_add(Dlo, Dll, K, KL)
            h, l = dd_add(h, l, -qhi, -qhil)
            h, l = dd_add(h, l, -phi, -phil)
            if h + l > S_SLACK:
                continue
            tw = (qhi - qlo) + (phi - plo) + (qhil - qlol) + (phil - plol) + 2 * S_SLACK
            if best < 0 or (Dhi - Dlo) + (Dhl - Dll) <= tw:
                if nout < out.shape[0]:
                    out[nout, 0] = q
                    out[nout, 1] = qp
                    for i in range(nop):
                        out[nout, 2 + i] = a[i]
                        out[nout, 2 + nop + i] = fx[i]
                nout += 1
                continue
        # split the open component with the widest contribution
        if top + 2 > maxd:
            stats[3] += 1
            return nout
        for i in range(nop):
            st_a[top, i] = a[i]
            st_f[top, i] = fx[i]
        st_a[top, best] = a[best] + 1
        st_p[top] = qp
        top += 1
        if allowed[best, a[best]]:
            for i in range(nop):
                st_a[top, i] = a[i]
                st_f[top, i] = fx[i]
            st_f[top, best] = 1
            st_p[top] = qp
            top += 1
        stats[1] += 1
    return nout


@njit(cache=True)
```

```
def run_block(qs, K, KL, dtab, dtl, allowed, floor, amax, out, stats):
    nout = 0
    for j in range(qs.shape[0]):
        nout = run_q(qs[j], K, KL, dtab, dtl, allowed, floor, amax, out, nout, stats)
    return nout


def prepare(comps, search):
    """Tables for a node's open components: (opens, K, KL, dtab, dtl, allowed, floor, amax).

    K = log(2/hi) and d_p(a) = -log(1 - p^-(2a+1)) are summed exactly as
    rationals from their series (truncation below 1e-43) and split into
    double-double (high, low) pairs."""
    from fractions import Fraction as F
    lo, hi, _ = search.bounds(comps)

    def dd(x):
        h = float(x)
        return h, float(x - F(h))

    u = (2 - hi) / hi                       # K = log1p(u), u ~ 1.8e-9
    K, KL = dd(u - u**2 / 2 + u**3 / 3 - u**4 / 4 + u**5 / 5)
    opens = [(p, a) for p, a, fx in comps if not fx]
    amax = []
    for p, a in opens:
        m = a
        while p ** -(2 * (m + 1) + 1) > D_FLOOR:
            m += 1
        amax.append(m)
    A = max(amax) + 2
    dtab = np.zeros((len(opens), A))
    dtl = np.zeros((len(opens), A))
    allowed = np.zeros((len(opens), A), np.bool_)
    for i, (p, a0) in enumerate(opens):
        for a in range(a0, amax[i] + 1):
            t = F(1, p ** (2 * a + 1))      # t <= 7^-11 ~ 5e-10
            dtab[i, a], dtl[i, a] = dd(t + t**2 / 2 + t**3 / 3 + t**4 / 4 + t**5 / 5)
            allowed[i, a] = search.coprime_ok_fix(comps, p, a)
    floor = np.array([a for _, a in opens], np.int64)
    return opens, K, KL, dtab, dtl, allowed, floor, np.array(amax, np.int64)


def primes_between(a, b):
    """Primes in [a, b] by a segmented sieve."""
    import numpy as np
    r = int(math.isqrt(b)) + 1
    small = np.ones(r + 1, bool)
    small[:2] = False
    for i in range(2, int(r ** 0.5) + 1):
        if small[i]:
            small[i * i::i] = False
    ps = np.nonzero(small)[0]
    seg = np.ones(b - a + 1, bool)
    for p in ps:
        st = max(p * p, (a + p - 1) // p * p)
        seg[st - a::p] = False
    if a <= 1:
        seg[:2 - a] = False
    return (np.nonzero(seg)[0] + a).astype(np.int64)
```

1.9 qfast_verify.py

Settles qfast survivors with the exact search.

```
"""Settle qfast candidates exactly with the original Search code."""
from qp import Search

def candidate_comps(comps, opens, row):
    nop = len(opens)
    q, qq = int(row[0]), int(row[1])
    box = {p: (int(row[2 + i]), bool(row[2 + nop + i])) for i, (p, _) in enumerate(opens)}
    new = tuple((p, *box[p]) if p in box else (p, a, fx) for p, a, fx in comps)
    return new + ((q, 1, False), (qq, 1, False))

def verify(comps, opens, rows, exclude=(3, 5)):
    s = Search(c=1, exclude=exclude)
```

```
    for row in rows:
        s.search(candidate_comps(comps, opens, row), 0)
    return s
```

1.10 run_qfast.py

Runs qfast over the unresolved node.

```
"""Run qfast over the rest of the unresolved q-range node of a results file.

usage: python run_qfast.py RESULTS.jsonl [CHUNK]
Writes RESULTS.qfast.jsonl (one line per chunk, resumable) and, when every
chunk is in, settles any candidates with the exact code and writes
RESULTS.qfast.done.
"""
import ast, json, os, sys, time
import numpy as np
from qp import Search
import qfast, qfast_verify
from priority import raise_priority
print("priority", raise_priority(), flush=True)

path = sys.argv[1]
CHUNK = int(sys.argv[2]) if len(sys.argv) > 2 else 10_000_000
rec = [json.loads(l) for l in open(path) if l.strip()]
nodes = [ast.literal_eval(u) for r in rec for u in r["unresolved"]]
assert len(nodes) == 1 and nodes[0][0] == "q-range"
_, comps, r, qstop = nodes[0]
assert r == 2
s = Search(c=1, exclude={3, 5})
opens, K, KL, dtab, dtl, allowed, floor, amax = qfast.prepare(comps, s)
lo, hi, _ = s.bounds(comps)
# (3) at r = 2: hi (q/(q-1))^2 > 2 is necessary for the smaller prime q
n, d = hi.numerator, hi.denominator
sq = (2 * d / n) ** 0.5
qend = int(sq / (sq - 1)) + 10
while not n * qend * qend > 2 * d * (qend - 1) ** 2:
    qend -= 1
assert not n * (qend + 1) ** 2 > 2 * d * qend ** 2
qa = qstop + 1            # qstop itself was searched by the original run
out_path = path.replace(".jsonl", ".qfast.jsonl")
done = set()
if os.path.exists(out_path):
    for l in open(out_path):
        if l.strip():
            done.add(json.loads(l)["start"])
print(f"node r={r}, q from {qa} to {qend}, K={K:.6e}", flush=True)
out = np.zeros((1_000_000, 2 + 2 * len(opens)), np.int64)
for start in range(qa, qend + 1, CHUNK):
    if start in done:
        continue
    end = min(start + CHUNK - 1, qend)
    qs = qfast.primes_between(start, end)
    st = np.zeros(4, np.int64)
    t0 = time.time()
    m = qfast.run_block(qs, K, KL, dtab, dtl, allowed, floor, amax, out, st)
    cands = out[:min(m, len(out))].tolist()
    line = {"start": start, "end": end, "primes": len(qs), "states": int(st[0]),
            "unresolved": int(st[2]), "overflow": int(st[3]), "candidates": m,
            "cand_rows": cands, "seconds": round(time.time() - t0, 1)}
    with open(out_path, "a") as f:
        f.write(json.dumps(line) + "\n")
    print(start, end, len(qs), int(st[0]), m, int(st[2]), int(st[3]), line["seconds"], flush=True)

lines = [json.loads(l) for l in open(out_path) if l.strip()]
starts = sorted(l["start"] for l in lines)
assert starts == list(range(qa, qend + 1, CHUNK)), "chunks missing"
rows = [row for l in lines for row in l["cand_rows"]]
bad = sum(l["unresolved"] + l["overflow"] for l in lines) + sum(l["candidates"] > len(l["cand_rows"]) for l in lines)
v = qfast_verify.verify(comps, opens, rows)
summary = {"q_from": qa, "q_to": qend, "primes": sum(l["primes"] for l in lines),
           "states": sum(l["states"] for l in lines), "candidates": len(rows),
           "exact_nodes": v.nodes, "solutions": v.solutions,
           "unresolved": len(v.unresolved) + bad,
           "verdict": "NODE CLOSED" if not v.solutions and not v.unresolved and not bad else "NOT CLOSED"}
```

```
open(path.replace(".jsonl", ".qfast.done"), "w").write(json.dumps(summary) + "\n")
print(json.dumps(summary))
```

1.11 test_qfast.py

Planted-solution test for qfast.

```
"""Planted-solution test for qfast.run_q, then timing on the real node."""
import ast, json, math, random, time
import numpy as np
from qp import Search
import qfast

rec = [json.loads(l) for l in open('results/cond_3-5_17.jsonl')]
_, comps, r, qstop = ast.literal_eval(rec[-1]['unresolved'][0])
s = Search(c=1, exclude={3, 5})
opens, K, KL, dtab, dtl, allowed, floor, amax = qfast.prepare(comps, s)
from fractions import Fraction as F
print("opens", opens, "K", K, "amax", list(amax))
nop = len(opens)
out = np.zeros((200000, 2 + 2 * nop), np.int64)
everything = np.ones_like(allowed)
random.seed(1)
ok = 0
T = 300
for t in range(T):
    q = qfast.primes_between(random.randrange(571_300_000, 1_130_000_000), 0) if False else None
    # pick q, q' primes and a fully fixed exponent vector, then choose K to fit exactly
    while True:
        q = random.randrange(571_300_000, 1_130_000_000) | 1
        if qfast.isprime(q): break
    while True:
        qq = random.randrange(q + 2, q * (3 if t % 3 else 50)) | 1
        if qfast.isprime(qq): break
    a = [f + random.choice([0, 0, 1, 2, 5]) for f in floor]
    D = sum(F(dtab[i, a[i]]) + F(dtl[i, a[i]]) for i in range(nop))
    b1 = qfast.log_f_bounds(q); b2 = qfast.log_f_bounds(qq)
    Ke = F(b1[0]) + F(b1[1]) + F(b2[0]) + F(b2[1]) - D
    Kp = float(Ke); KpL = float(Ke - F(Kp))
    stats = np.zeros(4, np.int64)
    n = qfast.run_q(q, Kp, KpL, dtab, dtl, everything, floor, amax, out, 0, stats)
    hit = any(out[j, 1] == qq and all((out[j, 2 + i] == a[i]) if out[j, 2 + nop + i] else (out[j, 2 + i] <= a[i]) for i in
    ↪  range(nop)) for j in range(min(n, len(out))))
    ok += hit
    if not hit:
        print("MISS", q, qq, a, n, list(stats))
print(f"planted: {ok}/{T} found")

qs = qfast.primes_between(571_287_684, 571_287_684 + 20000)
stats = np.zeros(4, np.int64)
t0 = time.time()
n = qfast.run_block(qs[:5], K, KL, dtab, dtl, allowed, floor, amax, out, stats)   # compile
t0 = time.time(); stats[:] = 0
n = qfast.run_block(qs, K, KL, dtab, dtl, allowed, floor, amax, out, stats)
dt = time.time() - t0
nf = sum(all(out[j, 2 + nop:]) for j in range(min(n, len(out))))
print("all-fixed candidates", nf)
print(f"{len(qs)} q in {dt:.2f}s ({dt/len(qs)*1e3:.2f} ms/q), states {stats[0]}, splits {stats[1]}, unresolved {stats[2]},
↪  overflow {stats[3]}, candidates {n}")
```

1.12 spot_qrange.py

Spot check of the qfast node by the exact search.

```
"""Spot check of the qfast-closed node with the original exact code.

Runs the body of Search.enumerate_next on N randomly sampled primes q from the range qfast
closed (one q at a time), in parallel.
Independent of qfast: shares only qp.py. Agreement is necessary, not sufficient.
usage: python spot_qrange.py N WORKERS
"""
import ast, json, random, sys, time
from multiprocessing import Pool
from qp import Search, next_prime

def one(q):
```

```
    rec = [json.loads(l) for l in open('results/cond_3-5_17.jsonl')]
    _, comps, r, _ = ast.literal_eval(rec[-1]['unresolved'][0])
    from fractions import Fraction as F
    s = Search(c=1, exclude={3, 5})
    lo, hi, n = s.bounds(comps)
    # the body of enumerate_next's loop for this one q
    if (lo * F(q * q + q + 1, q * q) <= 2 + F(1, n * q * q)
            and s.coprime_ok_add(comps, q)):
        s.search(comps + ((q, 1, False),), r - 1)
    return q, s.nodes, s.solutions, len(s.unresolved)

if __name__ == '__main__':
    from priority import raise_priority
    N, W = int(sys.argv[1]), int(sys.argv[2])
    random.seed(20261016)
    qs = sorted({next_prime(random.randrange(571_287_683, 1_137_878_224)) for _ in range(N)})
    t0 = time.time()
    with Pool(W, initializer=raise_priority) as p:
        res = p.map(one, qs, chunksize=4)
    out = {"sampled": len(res), "states": sum(r[1] for r in res),
           "solutions": [r[2] for r in res if r[2]], "unresolved": sum(r[3] for r in res),
           "q_min": qs[0], "q_max": qs[-1], "seconds": round(time.time() - t0)}
    open('results/cond_3-5_17.spot.json', 'w').write(json.dumps(out) + "\n")
    print(json.dumps(out))
```

1.13 qnode.py

Compiled search of any node with two primes left (omega = 7).

```
"""Compiled search for any node with two primes left to choose (r = 2).

Generalises qfast.py, which closed one such node for omega = 17. There the
last two primes were huge, so their own exponents could be ignored; here they
may be small (omega = 7 has nodes like 3^a 5^b 17^c 257^d q5^2 . q . q'), so
the exponents of q and q' are components of the box like any other.

Log form. Every component x with exponent 2b contributes
log f(x, b) = L(x) - d_x(b), with L(x) = -log(1 - 1/x) and
d_x(b) = -log(1 - x^-(2b+1)), so sigma(N) = 2N + c becomes

    L(q) + L(q') = K + sum_x d_x(b_x) + eps,   K = log 2 - sum_{node} L(p),
    0 < eps = log(1 + c/(2N)) <= c / (2 N_min).

A box gives each component an exponent either fixed (d exact) or open from a
floor (d in (0, d(floor)]). The search goes r = 2 (choose q), r = 1 (choose q'),
r = 0 (check), splitting the open component with the widest contribution when a
range or a box is too wide. All sums are double-double; node constants come from
exact rationals or 60-digit Decimal logs; every comparison carries slack S far
above the rounding error, so the search keeps too much and never too little.
Survivors are written out and settled by Search.search, the original exact code.

Prunes kept: the residue test when a node exponent is fixed (precomputed with
Search.coprime_ok_fix) and the mod-8 test when q's or q''s exponent is fixed.
Prunes dropped (coprimality with q and q', the residue test on q and q') only
shrink the tree, so dropping them is sound.
"""
import math
from decimal import Decimal, getcontext
from fractions import Fraction as F

import numpy as np
from numba import njit

from qp import is_probable_prime

S_SLACK = 1e-35
# Rounding error of a double-double chain grows with the size of its terms, not
# just with their result: about u^2 = 2^-106 per operation per unit of operand.
# With terms up to ~0.2 (L(q) for small q) a fixed 1e-35 is NOT enough, so every
# comparison also carries REL times the sum of the magnitudes it combines. REL is
# 2048 u^2, a deliberately loose cover for the worst chain (see the paper,
# "Soundness of the floating-point comparisons").
REL = 2048.0 * 2.0 ** -106
D_FLOOR = 1e-58          # bound on d for exponents past the table
W_Q2 = 32                # enumerate q' when the range holds at most this many integers
Q2_MAX = 4e18            # beyond this q' goes to the exact code (int64 limit)
```

```
W_LAST = 4096           # window allowed when q' is the only unknown left
SIEVE_SEG = 8192        # q is walked in sieved segments of this many integers

getcontext().prec = 80


# ------------------------------------------------------------------ double-double
@njit(cache=True)
def dd_add(ah, al, bh, bl):
    s = ah + bh
    v = s - ah
    e = (ah - (s - v)) + (bh - v) + al + bl
    h = s + e
    return h, e - (h - s)


@njit(cache=True)
def two_prod(a, b):
    p = a * b
    c = 134217729.0 * a
    ah = c - (c - a)
    al = a - ah
    c = 134217729.0 * b
    bh = c - (c - b)
    bl = b - bh
    return p, ((ah * bh - p) + ah * bl + al * bh) + al * bl


@njit(cache=True)
def dd_mul(ah, al, bh, bl):
    p, e = two_prod(ah, bh)
    e += ah * bl + al * bh
    h = p + e
    return h, e - (h - p)


@njit(cache=True)
def dd_recip(x):
    h = 1.0 / x
    p, pe = two_prod(h, x)
    r = (p - 1.0) + pe
    return h, -h * r


@njit(cache=True)
def neglog1m(xh, xl):
    """-log(1 - x) = sum x^k / k for 0 < x <= 1/7, double-double."""
    sh, sl = xh, xl
    ph, pl = xh, xl
    k = 1
    while True:
        k += 1
        ph, pl = dd_mul(ph, pl, xh, xl)
        t = ph / k
        if t < 1e-48:
            break
        # t as double-double: ph/k with its remainder
        th = t
        rh, rl = two_prod(th, float(k))
        tl = ((ph - rh) - rl + pl) / k
        sh, sl = dd_add(sh, sl, th, tl)
    return sh, sl


@njit(cache=True)
def L_of(x):
    yh, yl = dd_recip(float(x))
    return neglog1m(yh, yl)


@njit(cache=True)
def d_of(x, b):
    """d_x(b) = -log(1 - x^-(2b+1)) as double-double."""
    yh, yl = dd_recip(float(x))
    ph, pl = yh, yl
    for _ in range(2 * b):
```

```
        ph, pl = dd_mul(ph, pl, yh, yl)
    if ph < 1e-300:
        return 0.0, 0.0
    return neglog1m(ph, pl)


# ---------------------------------------------------------------- primes
@njit(cache=True)
def mulmod(a, b, n):
    return (((a * (b >> 19)) % n << 19) + a * (b & 524287)) % n


@njit(cache=True)
def isprime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in (2, 3, 5, 7, 11, 13, 17):
        x = 1
        base = a % n
        e = d
        while e:
            if e & 1:
                x = mulmod(x, base, n)
            base = mulmod(base, base, n)
            e >>= 1
        if x == 1 or x == n - 1:
            continue
        comp = True
        for _ in range(s - 1):
            x = mulmod(x, x, n)
            if x == n - 1:
                comp = False
                break
        if comp:
            return False
    return True


@njit(cache=True)
def mod8_ok(p, b):
    # sigma(p^2b) = 1 + b(p+1) mod 8 must be 1 or 3 (residue lemma, c = 1 only)
    r = (1 + b * (p + 1)) % 8
    return r == 1 or r == 3


# ---------------------------------------------------------------- kernel
@njit(cache=True)
def box_sum(n0, a, fx, dtab, dtl, amax):
    """Node part of the box: (Dlo, Dlo_l, Dhi, Dhi_l, widest index, width, lnN floor)."""
    Dlo = 0.0
    Dll = 0.0
    Dhi = 0.0
    Dhl = 0.0
    best = -1
    bw = 0.0
    for i in range(n0):
        if a[i] > amax[i]:
            Dhi, Dhl = dd_add(Dhi, Dhl, D_FLOOR, 0.0)
            continue
        d = dtab[i, a[i]]
        dl = dtl[i, a[i]]
        Dhi, Dhl = dd_add(Dhi, Dhl, d, dl)
        if fx[i]:
            Dlo, Dll = dd_add(Dlo, Dll, d, dl)
        elif d > bw:
            bw = d
            best = i
    return Dlo, Dll, Dhi, Dhl, best, bw
```

```
@njit(cache=True)
def run_node(n0, primes, floor0, fixed0, dtab, dtl, allowed, amax, lnp,
             K, KL, c, use_mod8, last, sp, out, stats, qfrom=0, qto=1 << 62):
    """Search one r = 2 node, for the smaller prime q in [qfrom, qto) only (so a
    dear node can be split across processes). Candidates go to out as rows:
    q, q', bq, fq, bq', fq', a_0..a_{n0-1}, fx_0..fx_{n0-1}. Returns count."""
    maxd = 1 << 14
    # stack: level (2,1,0), node a/fx, q, bq, fq, q', bq', fq'
    st_lv = np.empty(maxd, np.int64)
    st_a = np.empty((maxd, n0), np.int64)
    st_f = np.empty((maxd, n0), np.int64)
    st_q = np.empty((maxd, 6), np.int64)
    top = 0
    st_lv[0] = 2
    for i in range(n0):
        st_a[0, i] = floor0[i]
        st_f[0, i] = fixed0[i]
    for j in range(6):
        st_q[0, j] = 0
    top = 1
    nout = 0
    a = np.empty(n0, np.int64)
    fx = np.empty(n0, np.int64)
    seg = np.empty(SIEVE_SEG, np.bool_)
    while top > 0:
        top -= 1
        stats[0] += 1
        lv = st_lv[top]
        for i in range(n0):
            a[i] = st_a[top, i]
            fx[i] = st_f[top, i]
        if lv == 3:
            # cursor over the q-range: sieve one segment, leave a cursor for the rest
            stats[0] -= 1
            lo_s = st_q[top, 0]
            qmax = st_q[top, 1]
            hi_s = min(lo_s + SIEVE_SEG - 1, qmax)
            if hi_s < qmax:
                st_q[top, 0] = hi_s + 1
                top += 1                          # cursor stays, primes go above it
            for j in range(hi_s - lo_s + 1):
                seg[j] = True
            for k in range(sp.shape[0]):
                p = sp[k]
                if p * p > hi_s:
                    break
                st = max(p * p, ((lo_s + p - 1) // p) * p)
                for m in range(st, hi_s + 1, p):
                    seg[m - lo_s] = False
            for j in range(hi_s - lo_s, -1, -1):  # push high first, so low q runs first
                qq = lo_s + j
                if seg[j] and qq > 2:
                    if top >= maxd:
                        stats[3] += 1
                        return nout
                    st_lv[top] = 1
                    for i in range(n0):
                        st_a[top, i] = a[i]
                        st_f[top, i] = fx[i]
                    st_q[top, 0] = qq
                    st_q[top, 1] = 1
                    st_q[top, 2] = 0
                    st_q[top, 3] = 0
                    st_q[top, 4] = 0
                    st_q[top, 5] = 0
                    top += 1
            continue
        q = st_q[top, 0]
        bq = st_q[top, 1]
        fq = st_q[top, 2]
        q2 = st_q[top, 3]
        bq2 = st_q[top, 4]
        fq2 = st_q[top, 5]
        Dlo, Dll, Dhi, Dhl, best, bw = box_sum(n0, a, fx, dtab, dtl, amax)
```

```
        lnN = 0.0
        for i in range(n0):
            lnN += 2.0 * a[i] * lnp[i]
        lnM = lnN                               # node part only
        # widest open among q, q' exponents (index n0 = q, n0 + 1 = q')
        if lv <= 1:
            dh, dl = d_of(q, bq)
            Dhi, Dhl = dd_add(Dhi, Dhl, dh, dl)
            if fq:
                Dlo, Dll = dd_add(Dlo, Dll, dh, dl)
            elif dh > bw:
                bw = dh
                best = n0
            lnN += 2.0 * bq * math.log(q)
        if lv == 0:
            dh, dl = d_of(q2, bq2)
            Dhi, Dhl = dd_add(Dhi, Dhl, dh, dl)
            if fq2:
                Dlo, Dll = dd_add(Dlo, Dll, dh, dl)
            elif dh > bw:
                bw = dh
                best = n0 + 1
            lnN += 2.0 * bq2 * math.log(q2)
        # eps bound: c / (2 N_min), with q, q' at their least possible size
        if lv == 2:
            lnN += 4.0 * math.log(last + 1.0)
        elif lv == 1:
            lnN += 2.0 * math.log(q + 1.0)
        epsmax = 0.5 * c * math.exp(-lnN) * 1.000001
        split = False
        if lv == 2:
            # q <= q' gives 2 L(q) >= K + Dlo, so q <= qmax
            sl = S_SLACK + REL * (abs(K) + Dhi)
            h, l = dd_add(K, KL, Dlo, Dll)
            X = 0.5 * (h + l) - sl
            if X <= 0.0 and best >= 0:
                split = True
            else:
                if X <= 0.0:
                    # all fixed, node abundancy >= 2: only eps can balance, and
                    # 1/q < eps <= c/(2 M q^4) gives q^3 < c/(2M)
                    qmax = int((0.5 * c * math.exp(-lnM)) ** (1.0 / 3.0) * 1.000001) + 2
                else:
                    qmax = int(1.0 / (-math.expm1(-X))) + 2
                # q' > q gives log f(q,1) < K + Dhi + eps, so q >= qlow
                h, l = dd_add(K, KL, Dhi, Dhl)
                Y = h + l + epsmax + sl
                if Y <= 0.0:
                    continue
                if Y > 30.0:                        # huge eps (c > 1 only): no lower bound
                    qlow = last + 1
                else:
                    yy = math.expm1(Y)
                    qlow = max(int((1.0 + math.sqrt(1.0 + 4.0 * yy)) / (2.0 * yy)) - 2, last + 1)
                qlow = max(qlow, qfrom)
                qmax = min(qmax, qto)
                if qmax < qlow:
                    continue
                if best >= 0 and qmax > 5000 and qmax > 8 * qlow:
                    split = True
                elif qmax >= (1 << 38):
                    stats[2] += 1
                    continue
                else:
                    # walk primes q in [qlow, qmax] by a cursor state
                    if top >= maxd:
                        stats[3] += 1
                        return nout
                    st_lv[top] = 3
                    for i in range(n0):
                        st_a[top, i] = a[i]
                        st_f[top, i] = fx[i]
                    st_q[top, 0] = qlow
                    st_q[top, 1] = qmax
                    top += 1
                    continue
```

```
        elif lv == 1:
            Lh, Ll = L_of(q)
            sl = S_SLACK + REL * (abs(K) + Dhi + Lh)
            # R = L(q') - d_q' = K + D + eps - L(q); q' needs log f(q',1) <= Rhi, L(q') > Rlo
            h, l = dd_add(K, KL, Dhi, Dhl)
            h, l = dd_add(h, l, -Lh, -Ll)
            Rhi = h + l + epsmax + sl
            h, l = dd_add(K, KL, Dlo, Dll)
            h, l = dd_add(h, l, -Lh, -Ll)
            Rlo = h + l - sl
            if Rhi <= 0.0:
                continue
            # bounds as floats first: q' can be far beyond int64 when R is tiny
            if Rhi > 30.0:                        # huge eps (c > 1 only): no lower bound
                flo = 0.0
            else:
                yy = math.expm1(Rhi)
                flo = (1.0 + math.sqrt(1.0 + 4.0 * yy)) / (2.0 * yy)
            if Rlo <= 0.0 and best < 0:
                # all fixed and K + D - L(q) <= 0: 1/q' < R <= eps <= c/(2 N1 q'^2)
                fhi = 0.5 * c * math.exp(-(lnM + 2.0 * bq * math.log(q))) * 1.000001
            elif Rlo <= 0.0:
                fhi = 1e300
            else:
                fhi = 1.0 / (-math.expm1(-Rlo))
            if fhi < flo - 4.0:
                continue
            if fhi > Q2_MAX and best >= 0:
                split = True
                lo_i = 0
                hi_i = 0
            elif fhi > Q2_MAX:
                lo_i = 0
                hi_i = 1 << 62                    # forces the hand-off below
            else:
                # doubles carry ~16 digits: widen by a relative margin as well
                lo_i = max(int(flo * (1.0 - 4e-15)) - 2, q + 1)
                hi_i = int(fhi * (1.0 + 4e-15)) + 2
                if hi_i < lo_i:
                    continue
            if split:
                pass
            elif hi_i - lo_i > W_Q2 and best >= 0:
                split = True
            elif hi_i - lo_i > W_LAST:
                # nothing left open but q': hand the node to the exact code,
                # whose last-prime routine solves for q' directly (q' = 0 row)
                if nout < out.shape[0]:
                    out[nout, 0] = q
                    out[nout, 1] = 0
                    out[nout, 2] = bq
                    out[nout, 3] = fq
                    out[nout, 4] = 0
                    out[nout, 5] = 0
                    for i in range(n0):
                        out[nout, 6 + i] = a[i]
                        out[nout, 6 + n0 + i] = fx[i]
                nout += 1
                stats[4] += 1
                continue
            else:
                for qq in range(lo_i, hi_i + 1):
                    # below 2^38 test primality here; above it the window is a
                    # few integers wide and the exact code tests any survivor
                    if qq % 2 == 1 and (qq >= (1 << 38) or isprime(qq)):
                        if top >= maxd:
                            stats[3] += 1
                            return nout
                        st_lv[top] = 0
                        for i in range(n0):
                            st_a[top, i] = a[i]
                            st_f[top, i] = fx[i]
                        st_q[top, 0] = q
                        st_q[top, 1] = bq
                        st_q[top, 2] = fq
                        st_q[top, 3] = qq
```

```
                        st_q[top, 4] = 1
                        st_q[top, 5] = 0
                        top += 1
                continue
        else:
            Lh, Ll = L_of(q)
            L2h, L2l = L_of(q2)
            sl = S_SLACK + REL * (abs(K) + Dhi + Lh + L2h)
            h, l = dd_add(Lh, Ll, L2h, L2l)
            # need K + Dlo <= L + L' <= K + Dhi + eps
            u, ul = dd_add(h, l, -K, -KL)
            v, vl = dd_add(u, ul, -Dlo, -Dll)
            if v + vl < -sl:
                continue
            v, vl = dd_add(u, ul, -Dhi, -Dhl)
            if v + vl > epsmax + sl:
                continue
            if best < 0 or (Dhi - Dlo) + (Dhl - Dll) <= epsmax + 2 * sl:
                if nout < out.shape[0]:
                    out[nout, 0] = q
                    out[nout, 1] = q2
                    out[nout, 2] = bq
                    out[nout, 3] = fq
                    out[nout, 4] = bq2
                    out[nout, 5] = fq2
                    for i in range(n0):
                        out[nout, 6 + i] = a[i]
                        out[nout, 6 + n0 + i] = fx[i]
                nout += 1
                continue
            split = True
        if not split:
            continue
        # split the widest open component: raised child, then fixed child
        if top + 2 > maxd:
            stats[3] += 1
            return nout
        stats[1] += 1
        for rep in range(2):
            st_lv[top] = lv
            for i in range(n0):
                st_a[top, i] = a[i]
                st_f[top, i] = fx[i]
            st_q[top, 0] = q
            st_q[top, 1] = bq
            st_q[top, 2] = fq
            st_q[top, 3] = q2
            st_q[top, 4] = bq2
            st_q[top, 5] = fq2
            if best < n0:
                if rep == 0:
                    st_a[top, best] = a[best] + 1
                else:
                    if not allowed[best, a[best]]:
                        break
                    st_f[top, best] = 1
            elif best == n0:
                if rep == 0:
                    st_q[top, 1] = bq + 1
                else:
                    if use_mod8 and not mod8_ok(q, bq):
                        break
                    st_q[top, 2] = 1
            else:
                if rep == 0:
                    st_q[top, 4] = bq2 + 1
                else:
                    if use_mod8 and not mod8_ok(q2, bq2):
                        break
                    st_q[top, 5] = 1
            top += 1
    return nout


# ---------------------------------------------------------------- Python side
def _dec_neglog1m(x):
```

```
    """-log(1 - x) for a Fraction 0 < x < 1, to 80 digits."""
    return -(Decimal(1) - Decimal(x.numerator) / Decimal(x.denominator)).ln()


def _dd(dec):
    h = float(dec)
    return h, float(dec - Decimal(h))


_SP = None


def small_primes():
    global _SP
    if _SP is None:
        n = 1 << 20
        s = np.ones(n, bool)
        s[:2] = False
        for i in range(2, int(n ** 0.5) + 1):
            if s[i]:
                s[i * i::i] = False
        _SP = np.nonzero(s)[0].astype(np.int64)
    return _SP


def prepare(comps, search):
    n0 = len(comps)
    amax = []
    for p, a, fx in comps:
        m = a
        if not fx:
            while F(1, p ** (2 * (m + 1) + 1)) > F(1, 10 ** 60):
                m += 1
        amax.append(m)
    A = max(amax) + 2
    dtab = np.zeros((n0, A))
    dtl = np.zeros((n0, A))
    allowed = np.zeros((n0, A), np.bool_)
    for i, (p, a0, fx) in enumerate(comps):
        for a in range(a0, amax[i] + 1):
            dtab[i, a], dtl[i, a] = _dd(_dec_neglog1m(F(1, p ** (2 * a + 1))))
            allowed[i, a] = search.coprime_ok_fix(comps, p, a)
    Ksum = Decimal(2).ln()
    for p, _, _ in comps:
        Ksum -= _dec_neglog1m(F(1, p))
    K, KL = _dd(Ksum)
    return dict(n0=n0, primes=np.array([p for p, _, _ in comps], np.int64),
                floor0=np.array([a for _, a, _ in comps], np.int64),
                fixed0=np.array([1 if fx else 0 for _, _, fx in comps], np.int64),
                dtab=dtab, dtl=dtl, allowed=allowed, amax=np.array(amax, np.int64),
                lnp=np.array([math.log(p) for p, _, _ in comps]), K=K, KL=KL)


def search_node(comps, search, cap=1_000_000, qfrom=0, qto=1 << 62):
    """Run one r = 2 node; settle survivors exactly. Returns (kernel_stats, rows)."""
    P = prepare(comps, search)
    last = max(p for p, _, _ in comps)
    n0 = P["n0"]
    out = np.zeros((cap, 6 + 2 * n0), np.int64)
    stats = np.zeros(5, np.int64)
    use_mod8 = 1 if search.c == 1 else 0
    m = run_node(n0, P["primes"], P["floor0"], P["fixed0"], P["dtab"], P["dtl"],
                 P["allowed"], P["amax"], P["lnp"], P["K"], P["KL"], float(search.c),
                 use_mod8, last, small_primes(), out, stats, qfrom, qto)
    rows = out[:min(m, cap)]
    if m > cap or stats[2] or stats[3]:
        search.unresolved.append(("qnode", comps, (qfrom, qto), int(m), stats.tolist()))
    for row in rows:
        q, q2, bq, fq, bq2, fq2 = (int(v) for v in row[:6])
        if q in search.exclude or q2 in search.exclude:
            continue
        new = tuple((p, int(row[6 + i]), bool(row[6 + n0 + i]))
                    for i, (p, _, _) in enumerate(comps))
        if q2 >= (1 << 38) and not is_probable_prime(q2):
            continue                              # kernel only tests primality below 2^38
        if q2 == 0:                               # q' left to Search's last_prime
```

```
            search.search(new + ((q, bq, bool(fq)),), 1)
        else:
            search.search(new + ((q, bq, bool(fq)), (q2, bq2, bool(fq2))), 0)
    return stats, len(rows)


class NodeSearch:
    """Mixin: route every r = 2 enumeration through the compiled kernel."""

    def enumerate_next(self, comps, r, lo, hi, nmin, last):
        if r != 2:
            return super().enumerate_next(comps, r, lo, hi, nmin, last)
        st, n = search_node(comps, self)
        self.kstates = getattr(self, "kstates", 0) + int(st[0])
        self.kcands = getattr(self, "kcands", 0) + n
```

1.14 run_qnode.py

Full omega = K run with the node kernel.

```
"""Full omega = K search with every r = 2 node done by the compiled kernel (qnode).

usage: python run_qnode.py K DEPTH WORKERS
Splits the tree at DEPTH components (run_parallel.Splitter), runs each subtree
with NodeSearch, appends one record per subtree to results/qnode_omega{K}_d{DEPTH}.jsonl
(resumable), and writes a .done summary when every subtree is in.
"""
import json, os, sys, time
from multiprocessing import Pool
from priority import raise_priority
from qp import Search
from run_parallel import Splitter, key
import qnode


class NS(qnode.NodeSearch, Search):
    pass


def run_task(task):
    prio = raise_priority()
    comps, r = task
    t0, c0 = time.time(), time.process_time()
    s = NS(c=1)
    s.search(tuple(tuple(c) for c in comps), r)
    return {"task": key(task), "nodes": s.nodes, "kernel_states": getattr(s, "kstates", 0),
            "kernel_cands": getattr(s, "kcands", 0), "leaf_checks": s.leaf_checks,
            "solutions": [str(n) for n in s.solutions],
            "unresolved": [repr(u) for u in s.unresolved],
            "seconds": round(time.time() - t0, 2), "cpu_seconds": round(time.process_time() - c0, 2),
            "priority_ok": prio["priority"]}


def main():
    k, depth, workers = int(sys.argv[1]), int(sys.argv[2]), int(sys.argv[3])
    # optional @file of subtree keys to leave out (run separately by qchunks.py)
    skip = set()
    if len(sys.argv) > 4 and sys.argv[4].startswith("@"):
        skip = {l.strip() for l in open(sys.argv[4][1:]) if l.strip()}
    sp = Splitter(depth)
    sp.run(k)
    head = {"nodes": sp.nodes, "solutions": sp.solutions, "unresolved": sp.unresolved}
    tasks = sp.tasks
    path = f"results/qnode_omega{k}_d{depth}.jsonl"
    done = {}
    if os.path.exists(path):
        for line in open(path):
            if line.strip():
                rec = json.loads(line)
                done[rec["task"]] = rec
    todo = [t for t in tasks if key(t) not in done and key(t) not in skip]
    print(f"omega={k} depth={depth}: {len(tasks)} subtrees, {len(done)} done, {len(todo)} to run "
          f"on {workers} workers; head {head}", flush=True)
    t0 = time.time()
    with open(path, "a") as out, Pool(workers) as pool:
        for i, rec in enumerate(pool.imap_unordered(run_task, todo, chunksize=4), 1):
```

```
            out.write(json.dumps(rec) + "\n")
            out.flush()
            done[rec["task"]] = rec
            if rec["solutions"] or rec["unresolved"]:
                print("FLAG", rec, flush=True)
            if i % 2000 == 0 or i == len(todo):
                print(f"  {len(done)}/{len(tasks)} subtrees, {time.time() - t0:.0f}s", flush=True)
    if skip:
        print(f"DONE except {len(skip)} skipped subtrees; combine with finalize_omega7.py", flush=True)
        return
    recs = [done[key(t)] for t in tasks]
    summary = {"omega": k, "depth": depth, "subtrees": len(tasks),
               "python_states": head["nodes"] + sum(r["nodes"] for r in recs),
               "kernel_states": sum(r["kernel_states"] for r in recs),
               "kernel_candidates": sum(r["kernel_cands"] for r in recs),
               "solutions": head["solutions"] + [x for r in recs for x in r["solutions"]],
               "unresolved": len(head["unresolved"]) + sum(len(r["unresolved"]) for r in recs),
               "core_seconds": round(sum(r["cpu_seconds"] for r in recs))}
    summary["verdict"] = "ELIMINATED" if not summary["solutions"] and not summary["unresolved"] else "NOT ELIMINATED"
    open(path.replace(".jsonl", ".done"), "w").write(json.dumps(summary) + "\n")
    print("DONE", json.dumps(summary), flush=True)


if __name__ == "__main__":
    main()
```

1.15 qchunks.py

Splits dear nodes by the range of q.

```
"""Split dear r = 2 nodes by the range of their smaller prime q and run the pieces in parallel.

usage: python qchunks.py OUT.jsonl CHUNKS WORKERS TASK_JSON [TASK_JSON ...]
Each TASK_JSON is a subtree key from the omega run, e.g.
'[[[3, 5, false], ..., [65539, 1, false]], 2]'. The subtree is expanded by
Search down to its r = 2 nodes, and each r = 2 node's q-range [last+1, 2^38)
is cut at CHUNKS geometric points; every piece runs as its own job. Coverage:
the pieces of each node tile [last+1, 2^38) exactly, and q >= 2^38 is flagged
by the kernel itself as unresolved. Resumable by (node, piece).
"""
import json, math, os, sys, time
from multiprocessing import Pool
from priority import raise_priority
from qp import Search
import qnode


class Collect(Search):
    """Expand a subtree down to its r = 2 nodes without searching them."""
    def __init__(self):
        super().__init__(c=1)
        self.nodes2 = []

    def enumerate_next(self, comps, r, lo, hi, nmin, last):
        if r == 2:
            self.nodes2.append(comps)
            return
        return super().enumerate_next(comps, r, lo, hi, nmin, last)


class NS(qnode.NodeSearch, Search):
    pass


def piece(job):
    raise_priority()
    comps, qa, qb = job
    comps = tuple(tuple(c) for c in comps)
    s = NS(c=1)
    t0, c0 = time.time(), time.process_time()
    st, n = qnode.search_node(comps, s, qfrom=qa, qto=qb)
    return {"node": json.dumps(comps), "qa": qa, "qb": qb, "kernel_states": int(st[0]),
            "cands": n, "exact_nodes": s.nodes, "solutions": [str(x) for x in s.solutions],
            "unresolved": [repr(u) for u in s.unresolved],
            "seconds": round(time.time() - t0, 1), "cpu_seconds": round(time.process_time() - c0, 1)}

```

```

def jobs_for(task, chunks):
    comps, r = json.loads(task)
    c = Collect()
    c.search(tuple(tuple(x) for x in comps), r)
    head = {"nodes": c.nodes, "solutions": c.solutions, "unresolved": c.unresolved}
    out = []
    for node in c.nodes2:
        lo = max(p for p, _, _ in node) + 1
        hi = 1 << 38
        cuts = sorted({int(round(lo * (hi / lo) ** (i / chunks))) for i in range(chunks + 1)})
        cuts[0], cuts[-1] = lo, hi
        for a, b in zip(cuts, cuts[1:]):
            out.append((node, a, b - 1))
    return head, out


if __name__ == "__main__":
    path, chunks, workers = sys.argv[1], int(sys.argv[2]), int(sys.argv[3])
    tasks = sys.argv[4:]
    if len(tasks) == 1 and tasks[0].startswith("@"):        # @file: one task per line
        tasks = [l.strip() for l in open(tasks[0][1:]) if l.strip()]
    jobs, heads = [], []
    for t in tasks:
        h, j = jobs_for(t, chunks)
        heads.append(h)
        jobs += j
    done = set()
    if os.path.exists(path):
        for l in open(path):
            if l.strip():
                r = json.loads(l)
                done.add((r["node"], r["qa"], r["qb"]))
    todo = [j for j in jobs if (json.dumps(tuple(tuple(c) for c in j[0])), j[1], j[2]) not in done]
    # dearest-looking first: small q pieces of each node
    print(f"{len(tasks)} subtrees -> {len(jobs)} pieces, {len(done)} done; heads {heads}", flush=True)
    t0 = time.time()
    with open(path, "a") as f, Pool(workers, initializer=raise_priority) as pool:
        for i, rec in enumerate(pool.imap_unordered(piece, todo, chunksize=1), 1):
            f.write(json.dumps(rec) + "\n")
            f.flush()
            if rec["solutions"] or rec["unresolved"]:
                print("FLAG", rec, flush=True)
            if i % 50 == 0 or i == len(todo):
                print(f"  {i}/{len(todo)} pieces, {time.time()-t0:.0f}s", flush=True)
    print("DONE", flush=True)
```

1.16 finalize_omega7.py

Combines the omega = 7 records and checks coverage.

```
"""Combine the omega = 7 qnode run and the straggler pieces into one verdict, checking coverage."""
import json
from run_parallel import Splitter, key
from qchunks import jobs_for

sp = Splitter(5)
sp.run(7)
main = {}
for l in open('results/qnode_omega7_d5.jsonl'):
    if l.strip():
        r = json.loads(l)
        main[r['task']] = r
strag = [l.strip() for l in open('results/stragglers.txt') if l.strip()]
pieces = {}
for l in open('results/qchunks_omega7.jsonl'):
    if l.strip():
        r = json.loads(l)
        pieces[(r['node'], r['qa'], r['qb'])] = r

missing, sols, unres, core = [], [], 0, 0.0
pstates = ksstates = 0
heads_nodes = 0
for t in sp.tasks:
    k = key(t)
    if k in strag:
```

```
        head, jobs = jobs_for(k, 600)
        heads_nodes += head['nodes']
        sols += head['solutions']
        unres += len(head['unresolved'])
        for node, a, b in jobs:
            r = pieces.get((json.dumps(tuple(tuple(c) for c in node)), a, b))
            if r is None:
                missing.append((k, a, b))
                continue
            sols += r['solutions']
            unres += len(r['unresolved'])
            core += r['cpu_seconds']
            ksstates += r['kernel_states']
            pstates += r['exact_nodes']
    elif k in main:
        r = main[k]
        sols += r['solutions']
        unres += len(r['unresolved'])
        core += r['cpu_seconds']
        ksstates += r['kernel_states']
        pstates += r['nodes']
    else:
        missing.append((k,))
summary = {"omega": 7, "subtrees": len(sp.tasks), "head_states": sp.nodes,
           "straggler_head_states": heads_nodes, "exact_states": pstates,
           "kernel_states": ksstates, "core_hours": round(core / 3600, 1),
           "solutions": sols, "unresolved": unres, "missing": len(missing)}
summary["verdict"] = ("ELIMINATED" if not sols and not unres and not missing and not sp.solutions
                      and not sp.unresolved else "NOT ELIMINATED")
open('results/omega7_final.json', 'w').write(json.dumps(summary) + "\n")
print(json.dumps(summary))
if missing:
    print(missing[:5])
```

1.17 test_qnode.py

Node kernel against brute-force solutions and omega <= 6.

```
"""NodeSearch (compiled r = 2 kernel) against Search, on real solutions of
sigma(n) = 2n + c and on the omega <= 6 reproduction."""
import sys, time
from qp import Search
import qnode

class NS(qnode.NodeSearch, Search):
    pass

mode = sys.argv[1] if len(sys.argv) > 1 else "c"
if mode == "c":
    CMAX = int(sys.argv[2]) if len(sys.argv) > 2 else 3000
    W = [int(x) for x in sys.argv[3].split(",")] if len(sys.argv) > 3 else [3, 4]
    bad = sols = 0; t0 = time.time()
    for w in W:
        for c in range(1, CMAX + 1):
            a = Search(c=c); a.run(w)
            b = NS(c=c); b.run(w)
            sols += len(a.solutions)
            if sorted(a.solutions) != sorted(b.solutions) or bool(a.unresolved) != bool(b.unresolved):
                bad += 1
                print("MISMATCH", c, w, a.solutions, b.solutions, a.unresolved[:1], b.unresolved[:1], flush=True)
    print(f"{bad} mismatches, {sols} solutions compared, {time.time()-t0:.0f}s")
elif mode == "w":
    for w in range(1, int(sys.argv[2]) + 1):
        t0 = time.time()
        b = NS(c=1); b.run(w)
        print(w, b.nodes, getattr(b, 'kstates', 0), getattr(b, 'kcands', 0), b.solutions, len(b.unresolved),
        ↪  f"{time.time()-t0:.1f}s", flush=True)

if mode == "b":
    # every (c, omega) with a brute-force solution n = k^2, k <= K, c <= CMAX
    from collections import defaultdict
    from qp import factorize, sig
    K = int(sys.argv[2]); CMAX = int(sys.argv[3])
    brute = defaultdict(list)
    for k in range(3, K + 1, 2):
        f = factorize(k); n = k * k; s = 1
```

```
        for p, e in f.items():
            s *= sig(p, 2 * e)
        cc = s - 2 * n
        if 0 < cc <= CMAX:
            brute[(cc, len(f))].append(n)
    bad = 0; t0 = time.time(); found = 0
    print(f"{len(brute)} cases to run", flush=True)
    STEP = int(sys.argv[4]) if len(sys.argv) > 4 else 1
    for idx, ((cc, w), ns) in enumerate(sorted(brute.items())[::STEP]):
        t1 = time.time()
        b = NS(c=cc); b.run(w)
        if time.time() - t1 > 5:
            print(f"slow: c={cc} omega={w} {time.time()-t1:.0f}s", flush=True)
        if idx % 200 == 0:
            print(f"  {idx} cases, c up to {cc}, {time.time()-t0:.0f}s", flush=True)
        small = sorted(x for x in b.solutions if x <= K * K)
        ok = small == sorted(ns) and not b.unresolved
        found += len(ns)
        if not ok:
            bad += 1
            print("BAD", cc, w, ns, b.solutions, b.unresolved[:1], flush=True)
    print(f"{len(brute)} (c, omega) cases, {found} brute-force solutions, {bad} bad, {time.time()-t0:.0f}s")
```

2 Supporting tools

2.1 paper/reconstruct_variants.py

Rebuilds every source version from the transcript.

```
"""Reconstruct every version of every source file from the session transcript.

Why this exists: several intermediate versions of the solver were written,
tested, found wrong and corrected before ever being committed, so they are in
no git commit. Claude Code's own file-history backups do not hold them either
(the backup filenames are null for files created during the session). What does
survive is the transcript, which records the full text of every Write and the
exact old/new strings of every Edit.

Replaying those in order reconstructs each version exactly. The replay is
self-checking: if the final reconstructed state of a file matches the file now
on disk byte for byte, every earlier state in that chain is also faithful,
because each step is an exact textual operation. Any file whose final state does
not match is reported as UNVERIFIED and must not be trusted.

Usage: python reconstruct_variants.py SESSION.jsonl [...]   (chronological order)
Writes variants/<name>.vNN<suffix> and variants/MANIFEST.md.
"""
import hashlib
import json
import os
import sys

REPO = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", ".."))
OUT = os.path.join(os.path.dirname(__file__), "variants")

# Short labels tying a version to the issue it represents in the paper. Assigned
# by reading the reconstructed diffs, not from memory. Keyed by (file, version).
LABELS = {
    ("qp.py", 1): "**Issue 1, faulty**: `last_prime` bisects only past the maximum "
                  "and flags its rising branch as unresolved instead of solving it",
    ("qp.py", 2): "`record` no longer re-factorises to re-check; callers verify",
    ("qp.py", 3): "the `lo >= 2` branch enumerates its finite prime range (Issue 2)",
    ("qp.py", 4): "coprimality generalised from c = 1 to any c (Issue 2)",
    ("qp.py", 5): "docstring for the coprimality bound corrected",
    ("qp.py", 6): "faulty rising-branch code removed from `last_prime` (Issue 1)",
    ("qp.py", 7): "**Issue 1, fixed**: `poly_roots` bisects every monotone stretch",
    ("qp.py", 8): "records the current state, for the branch-sampling diagnostic",
    ("qp.py", 9): "**Issue 4, fixed**: refine open exponents before the last prime",
    ("qp.py", 10): "that threshold exposed as a parameter",
    ("qp.py", 11): "**Issue 5, rejected**: same rule at every r, ten times slower",
    ("qp.py", 12): "ratio parameter added to the rejected rule",
    ("qp.py", 13): "rejected rule adjusted",
    ("qp.py", 14): "**Issue 5, reverted** to the r = 1 rule only",
    ("qp.py", 15): "**Issue 8, faulty**: early refinement keyed on `hi` alone, "
                   "which does not decrease, so refinement never terminates. Not "
```

```
                  "runnable by itself: it uses a parameter added only in v16",
    ("qp.py", 16): "the same defect, now runnable: threshold exposed as a parameter",
    ("qp.py", 17): "**Issue 8, fixed**: keyed on both ends of the range, so the "
                   "condition eventually fails and the branch ends",
    ("test_qp.py", 2): "phase 2 appended; later truncated through the shell, "
                       "which is why this chain does not verify",
}


def rel(p):
    p = p.replace("/", os.sep)
    return os.path.relpath(os.path.abspath(p), REPO) if os.path.isabs(p) else p


def replay(paths):
    state, versions = {}, {}
    for path in paths:
        for line in open(path, encoding="utf-8"):
            d = json.loads(line)
            if d.get("type") != "assistant":
                continue
            ts = d.get("timestamp", "")
            for b in d["message"].get("content", []):
                if b.get("type") != "tool_use":
                    continue
                name, inp = b["name"], b.get("input", {})
                if name not in ("Write", "Edit") or "file_path" not in inp:
                    continue
                f = rel(inp["file_path"])
                if not f.endswith(".py"):
                    continue
                if name == "Write":
                    state[f] = inp.get("content", "")
                    op = "Write (full file)"
                else:
                    old, new = inp.get("old_string", ""), inp.get("new_string", "")
                    if f not in state or old not in state[f]:
                        versions.setdefault(f, []).append(
                            (ts, "Edit FAILED TO REPLAY", None))
                        continue
                    n = -1 if inp.get("replace_all") else 1
                    state[f] = state[f].replace(old, new, n) if n == 1 else \
                        state[f].replace(old, new)
                    op = "Edit"
                versions.setdefault(f, []).append((ts, op, state[f]))
    return state, versions


def main():
    state, versions = replay(sys.argv[1:])
    os.makedirs(OUT, exist_ok=True)
    lines = ["# Reconstructed source variants", "",
             "Each version below is the exact state of the file after one recorded",
             "Write or Edit, replayed from the session transcript. `verified` means the",
             "final replayed state matches the file currently on disk byte for byte,",
             "which makes the whole chain for that file faithful.", ""]
    for f in sorted(versions):
        base = os.path.basename(f)
        stem, ext = os.path.splitext(base)
        disk = os.path.join(REPO, f)
        final = state.get(f)
        ok = os.path.exists(disk) and open(disk, encoding="utf-8").read() == final
        lines += [f"## {f} — {'verified' if ok else '**UNVERIFIED**'}", ""]
        if not ok:
            lines += ["The replayed chain does not reproduce the file now on disk. The"
                      " usual cause is an edit made through the shell rather than the"
                      " editing tool: the transcript records the command but not the"
                      " resulting text, so the chain has a gap. The versions below are"
                      " still exact up to that gap, and the final row is the file as it"
                      " stands on disk.", ""]
        lines += ["| v | time (UTC) | operation | sha1 | label |",
                  "|---|---|---|---|---|"]
        for i, (ts, op, text) in enumerate(versions[f], 1):
            if text is None:
                lines.append(f"| {i} | {ts[:19]} | {op} | — | — |")
                continue
```

```
            name = f"{stem}.v{i:02d}{ext}"
            open(os.path.join(OUT, name), "w", encoding="utf-8", newline="\n").write(text)
            sha = hashlib.sha1(text.encode()).hexdigest()[:12]
            lines.append(f"| {i} | {ts[:19]} | {op} | `{sha}` | "
                         f"{LABELS.get((base, i), '')} |")
        if not ok and os.path.exists(disk):
            text = open(disk, encoding="utf-8").read()
            name = f"{stem}.current{ext}"
            open(os.path.join(OUT, name), "w", encoding="utf-8", newline="\n").write(text)
            sha = hashlib.sha1(text.encode()).hexdigest()[:12]
            lines.append(f"| current | — | file on disk | `{sha}` | "
                         f"the version actually used |")
        lines.append("")
        print(f"{f}: {len(versions[f])} versions, "
              f"{'verified' if ok else 'UNVERIFIED — do not trust'}")
    open(os.path.join(OUT, "MANIFEST.md"), "w", encoding="utf-8").write("\n".join(lines) + "\n")


if __name__ == "__main__":
    main()
```

2.2 paper/verify_variants.py

Runs each faulty version against its own defect.

```
"""Check that the faulty variants really do fail, and the fixed ones really pass.

Section~\\ref{sec:history} of the paper claims specific defects in specific
versions. A claim like that should be tested, not asserted, so this runs each
named variant against the defect it is supposed to exhibit:

  Issue 1  qp.v01 (before) must leave unresolved states on the planted
           last-prime cases; qp.v07 (after) must leave none.
  Issue 8  qp.v15 (before) must fail to terminate on omega = 5 within a
           generous node budget; qp.v17 (after) must finish in 193,559 nodes.

Each variant runs in its own temporary directory so nothing shadows the
working copy. Usage: python verify_variants.py
"""
import os
import shutil
import subprocess
import sys
import tempfile

HERE = os.path.dirname(os.path.abspath(__file__))
VAR = os.path.join(HERE, "variants")
PROJ = os.path.abspath(os.path.join(HERE, ".."))

LEAF_PROBE = r"""
import random, sys
from fractions import Fraction
from qp import Search, sig, is_probable_prime
# The same planted last-prime construction as test_leaves.py, fixed seed, so the
# two variants are asked exactly the same questions.
rng = random.Random(1)
odd = [p for p in range(3, 2000) if is_probable_prime(p)]
flags = recovered = tried = 0
while tried < 200:
    w = rng.randint(2, 5)
    ps = sorted(rng.sample(odd[:15], w - 1)); ps.append(rng.choice([p for p in odd if p > ps[-1]]))
    exps = [rng.choice([1, 1, 1, 2, 3, 4, 7, 12]) for _ in ps]
    n = s = 1
    for p, a in zip(ps, exps):
        n *= p ** (2 * a); s *= sig(p, 2 * a)
    c = s - 2 * n
    if c <= 0: continue
    m = sm = 1
    for p, a in zip(ps[:-1], exps[:-1]):
        m *= p ** (2 * a); sm *= sig(p, 2 * a)
    if sm >= 2 * m: continue
    tried += 1
    S = Search(c=c)
    S.last_prime(tuple((p, a, True) for p, a in zip(ps[:-1], exps[:-1])), Fraction(sm, m), m, ps[-2])
    if S.unresolved: flags += 1
    if n in S.solutions: recovered += 1
```

```
print(f"tried={tried} recovered={recovered} unresolved_cases={flags}")
"""

# The defect needs a state where hi is within about 2r/5000 of 2, which requires
# four Fermat primes: hi = 3/2 * 5/4 * 17/16 * 257/256 = 2 - 2^-15. Three primes
# reach only 255/128, so omega = 5 never triggers the rule at all and both
# versions behave identically there. The diagnostic for an infinite refinement
# path is the exponent floor of an open component growing without bound, since
# Lemma 8 (parity) bounds it in any terminating search.
# The whole subtree takes hours either way, so termination itself is not
# observable in a short run; that is what Proposition 2 is for. What IS observable
# is the pathology: an exponent floor climbing without bound. The tripwire below
# separates the two versions cleanly.
TERM_PROBE = r"""
from qp import Search
STATE = ((3, 1, False), (5, 1, False), (17, 1, False), (257, 1, False))
FLOOR_LIMIT = 400        # an exponent floor of 2*400 means 3^800: plainly runaway
NODE_BUDGET = 1_000_000
class Probe(Search):
    deepest = 0
    def search(self, comps, r):
        self.deepest = max(self.deepest, max((t[1] for t in comps if not t[2]), default=0))
        if self.deepest > FLOOR_LIMIT:
            raise SystemExit(f"RUNAWAY REFINEMENT: open exponent past 2*{FLOOR_LIMIT} "
                             f"after only {self.nodes:,} nodes")
        if self.nodes > NODE_BUDGET:
            raise SystemExit(f"no runaway: {self.nodes:,} nodes searched, deepest open "
                             f"exponent only 2*{self.deepest}, unresolved={len(self.unresolved)}")
        return super().search(comps, r)
s = Probe(c=1)
s.search(STATE, 2)
print(f"finished early: nodes={s.nodes} deepest=2*{s.deepest}")
"""


def run(variant, probe, label):
    d = tempfile.mkdtemp(prefix="qpvar_")
    try:
        shutil.copy(os.path.join(VAR, variant), os.path.join(d, "qp.py"))
        open(os.path.join(d, "probe.py"), "w").write(probe)
        p = subprocess.run([sys.executable, "probe.py"], cwd=d,
                           capture_output=True, text=True, timeout=1800)
        out = (p.stdout + p.stderr).strip().splitlines()
        print(f"  {variant:16s} {label:34s} -> {out[-1] if out else '(no output)'}")
    finally:
        shutil.rmtree(d, ignore_errors=True)


if __name__ == "__main__":
    print("Issue 1 - last-prime root finding (planted cases, same seed):")
    run("qp.v01.py", LEAF_PROBE, "before: expect unresolved > 0")
    run("qp.v07.py", LEAF_PROBE, "after:  expect unresolved = 0")
    print("Issue 8 - termination of refinement (the four-Fermat-prime state):")
    # v15 introduces the faulty rule but references a parameter that only appears
    # in v16, so v16 is the first runnable version carrying the defect.
    run("qp.v16.py", TERM_PROBE, "before: expect runaway refinement")
    run("qp.v17.py", TERM_PROBE, "after:  expect no runaway")
```

2.3 paper/export_code.py

Builds this appendix.

```
"""Build the code appendix (code.tex): current sources in full, plus the diffs
that introduced and removed each defect the paper describes.

Full listings of all 17 recovered solver versions would run to thousands of
lines and obscure the point. Instead this gives every current source in full,
then for each issue a unified diff between the version before and the version
after, which is exactly the claim the paper makes about that issue. The
complete set of versions is in paper/variants/ for anyone who wants to run one.

Usage: python export_code.py
"""
import difflib
import os

```

```
HERE = os.path.dirname(os.path.abspath(__file__))
PROJ = os.path.abspath(os.path.join(HERE, ".."))
VAR = os.path.join(HERE, "variants")

SOURCES = [
    ("qp.py", "The search: bounds, pruning and the two exact leaf solvers."),
    ("run_parallel.py", "Cuts the tree into independent subtrees and runs them."),
    ("split_task.py", "Splits one subtree further, for the case of Issue 7."),
    ("audit_results.py", "Audits a finished run for complete coverage."),
    ("test_qp.py", "Cross-check against brute force over odd squares."),
    ("test_leaves.py", "Planted solutions for the two closing lemmas."),
    ("peek.py", "Samples which states the search is visiting."),
    ("qfast.py", "Compiled search that closed the last omega = 17 node."),
    ("qfast_verify.py", "Settles qfast survivors with the exact search."),
    ("run_qfast.py", "Runs qfast over the unresolved node."),
    ("test_qfast.py", "Planted-solution test for qfast."),
    ("spot_qrange.py", "Spot check of the qfast node by the exact search."),
    ("qnode.py", "Compiled search of any node with two primes left (omega = 7)."),
    ("run_qnode.py", "Full omega = K run with the node kernel."),
    ("qchunks.py", "Splits dear nodes by the range of q."),
    ("finalize_omega7.py", "Combines the omega = 7 records and checks coverage."),
    ("test_qnode.py", "Node kernel against brute-force solutions and omega <= 6."),
]

# The conversation exporter is listed only in the private build; the public
# repository does not carry the conversation (QP_PUBLIC=1).
TOOLS = [
    ("paper/reconstruct_variants.py", "Rebuilds every source version from the transcript."),
    ("paper/verify_variants.py", "Runs each faulty version against its own defect."),
    ("paper/export_code.py", "Builds this appendix."),
]

if not os.environ.get("QP_PUBLIC"):
    TOOLS.insert(2, ("paper/export_conversation.py", "Builds the conversation appendix."))

DIFFS = [
    ("qp.v01.py", "qp.v07.py", "Issue 1: incomplete root finding, and its fix",
     "Before, the last-prime leaf bisects only the stretch beyond the maximum of "
     "$P_a$ and records an unresolved state for the rest. After, poly\\_roots "
     "locates the turning points from the single sign change of $P_a''$ and "
     "bisects every monotone stretch. Measured effect: 45 of 200 planted cases "
     "recovered and 190 flagged, against 200 recovered and none flagged."),
    ("qp.v14.py", "qp.v16.py", "Issue 8: the rule that does not terminate",
     "The added condition tests only \\texttt{hi}, which for an open component "
     "uses the limit $p/(p-1)$ and so does not change as the exponent bound "
     "rises. On the all-open path the condition therefore holds forever. "
     "Measured effect: an open exponent driven past $2 \\cdot 400$ after "
     "256,506 states."),
    ("qp.v16.py", "qp.v17.py", "Issue 8: the fix",
     "The condition now also tests the lower end of the admissible range, which "
     "does rise with the exponents, so it eventually fails. Measured effect: "
     "$10^6$ states on the same subtree with the deepest open exponent never "
     "exceeding $2 \\cdot 15$."),
]

HEAD = r"""\documentclass[9pt]{extarticle}
\usepackage[margin=0.75in]{geometry}
\usepackage{fontspec}
\setmonofont{DejaVuSansMono}[Extension=.ttf, Scale=0.82]
\usepackage{fancyvrb,fvextra,xcolor,hyperref}
\hypersetup{hidelinks}
\definecolor{add}{RGB}{20,110,40}
\definecolor{del}{RGB}{160,30,30}
\definecolor{hdr}{RGB}{90,90,90}
\newcommand{\difftitle}[1]{\par\medskip\noindent\textbf{#1}\par\smallskip}
\title{Appendix: source code}
\author{Ansel Marsh (an instance of Claude)}
\date{}
\begin{document}
\maketitle
\noindent Every source file as it currently stands, followed by the change that
introduced and the change that removed each defect discussed in the paper. The
seventeen recovered versions of the search itself are not listed in full --- they
would run to several thousand near-identical lines --- but all of them, with a
manifest of timestamps and hashes, are in \texttt{paper/variants/}, and each can
be run directly.
```

```
\tableofcontents
"""


def verbatim(text):
    # Verbatim needs no escaping; only guard against the end delimiter appearing.
    text = text.replace(r"\end {Verbatim}", r"\end {Verbatim}")
    return ("\\begin{Verbatim}[breaklines,breakanywhere,numbers=left,"
            "numbersep=4pt,fontsize=\\small]\n" + text.rstrip() + "\n\\end {Verbatim}\n")


def read(path):
    with open(path, encoding="utf-8") as f:
        return f.read()


def diff_block(a, b):
    lines = list(difflib.unified_diff(
        read(os.path.join(VAR, a)).splitlines(),
        read(os.path.join(VAR, b)).splitlines(),
        fromfile=a, tofile=b, lineterm="", n=2))
    out = ["\\begin{Verbatim}[breaklines,breakanywhere,commandchars=\\\\\\{\\},"
           "fontsize=\\small]"]
    for L in lines:
        L = L.replace("\\", "\\textbackslash{}").replace("{", "\\{").replace("}", "\\}")
        if L.startswith("+++") or L.startswith("---") or L.startswith("@@"):
            out.append(f"\\textcolor{{hdr}}{{{L}}}")
        elif L.startswith("+"):
            out.append(f"\\textcolor{{add}}{{{L}}}")
        elif L.startswith("-"):
            out.append(f"\\textcolor{{del}}{{{L}}}")
        else:
            out.append(L)
    out.append("\\end {Verbatim}")
    return "\n".join(out) + "\n"


def main():
    parts = [HEAD, r"\section{Current sources}"]
    for name, blurb in SOURCES:
        parts += [f"\\subsection{{\\texttt{{{name.replace('_', chr(92) + '_')}}}}}",
                  blurb, verbatim(read(os.path.join(PROJ, name)))]
    parts.append(r"\section{Supporting tools}")
    for name, blurb in TOOLS:
        base = name.replace("_", chr(92) + "_")
        parts += [f"\\subsection{{\\texttt{{{base}}}}}", blurb,
                  verbatim(read(os.path.join(PROJ, name)))]
    parts.append(r"\section{The changes that introduced and removed each defect}")
    for a, b, title, blurb in DIFFS:
        parts += [f"\\subsection{{{title}}}", blurb, "", diff_block(a, b)]
    parts.append(r"\end{document}")
    with open(os.environ.get("QP_CODE_OUT", os.path.join(HERE, "code.tex")), "w", encoding="utf-8") as f:
        f.write("\n".join(parts) + "\n")
    print("wrote code.tex")


if __name__ == "__main__":
    main()
```

3 The changes that introduced and removed each defect

3.1 Issue 1: incomplete root finding, and its fix

Before, the last-prime leaf bisects only the stretch beyond the maximum of P_a and records an unresolved state for the rest. After, poly_roots locates the turning points from the single sign change of P_a'' and bisects every monotone stretch. Measured effect: 45 of 200 planted cases recovered and 190 flagged, against 200 recovered and none flagged.

```
--- qp.v01.py
+++ qp.v07.py
@@ -16,6 +16,6 @@
   (3) Upper bound: the r remaining primes, all > last, contribute at most the
       product of q/(q-1) over the r consecutive primes after last.
-  (4) gcd(n, sigma(n)) = gcd(n, c): with c = 1 no prime of n divides sigma(p^(2a))
-      for any fixed component (applied only when c = 1).
+  (4) gcd(n, sigma(n)) = gcd(n, c), so a prime of n that divides sigma(p^(2a)) for a
```

```
+       fixed component must divide c. With c = 1: none may.
    (5) One open exponent, everything else fixed: sigma(n) = 2n + c = c (mod p^(2a))
        gives p^(2a) | sigma(n') + c(p - 1), where n' = n / p^(2a). Finite.
@@ -99,18 +99,17 @@
          return lo, hi, nmin

+     # (4): a prime of n dividing sigma(p^(2a)) must divide c
      def coprime_ok_fix(self, comps, p, a):
-         if self.c != 1:
-             return True
          s = sig(p, 2 * a)
-         return all(s % q for q, _, _ in comps if q != p)
+         return all(s % q or self.c % q == 0 for q, _, _ in comps if q != p)

      def coprime_ok_add(self, comps, q):
-         if self.c != 1:
+         if self.c % q == 0:
              return True
          return all(sig(p, 2 * a) % q for p, a, fx in comps if fx)

      def record(self, n):
-         if n > 1 and sigma_of(n) == 2 * n + self.c:
-             self.solutions.append(n)
+         # callers have already checked sigma(n) = 2n + c exactly from the factorisation
+         self.solutions.append(n)

      def run(self, k):
@@ -145,7 +144,10 @@
          if lo >= 2:
              # (1)+(2): lo(1 + 1/q) < 2 + c/(nmin q^2)  =>  q < c/(nmin lo)
-             if F(c) / (nmin * lo) <= last:
-                 return
-             self.unresolved.append(("lo>=2", comps, r))
+             qb = F(c) / (nmin * lo)
+             q = next_prime(last)
+             while q < qb:                       # never entered when c = 1
+                 if self.coprime_ok_add(comps, q):
+                     self.search(comps + ((q, 1, False),), r - 1)
+                 q = next_prime(q)
              return
          # (3)
@@ -211,26 +213,8 @@
                  break

-             def P(q):
-                 return q ** e * (2 * m - D * q) - c * q - (S - c)
-
-             peak = (e * 2 * m) // ((e + 1) * D)   # g rises below ~peak, falls above
-             # rising branch: prove no root, else scan it (flagged)
-             if qmin <= peak:
-                 g_lo = qmin ** e * (2 * m - D * qmin)
-                 if not g_lo > c * (peak + 1) + S:
-                     self.unresolved.append(("rising-branch", comps, a))
-             # falling branch: P strictly decreasing on [max(qmin, peak+1), qmax]
-             L = max(qmin, peak + 1)
-             R = qmax
-             if L <= R and P(L) >= 0 >= P(R):
-                 while L < R:
-                     mid = (L + R) // 2
-                     if P(mid) > 0:
-                         L = mid + 1
-                     else:
-                         R = mid
-                 if P(L) == 0:
-                     self.leaf_checks += 1
-                     self.record(m * L ** e)
+             for q in poly_roots(m, S, D, c, e, qmin, qmax):
+                 self.leaf_checks += 1
+                 if S * sig(q, e) == 2 * m * q ** e + c:
+                     self.record(m * q ** e)
              a += 1

@@ -262,4 +246,58 @@


+def poly_roots(m, S, D, c, e, lo, hi):
```

```
+    """All integers q in [lo, hi] with P(q) = q^e (2m - Dq) - cq - (S - c) = 0.
+
+    P'' = e q^(e-2) ((e-1) 2m - (e+1) D q) changes sign once, at q_inf, so P' rises
+    then falls and P has at most three monotone stretches. Find P' = 0 on each
+    side of q_inf by bisection, then bisect P on each stretch.
+    """
+    def P(q):
+        return q ** e * (2 * m - D * q) - c * q - (S - c)
+
+    def dP(q):
+        return e * 2 * m * q ** (e - 1) - (e + 1) * D * q ** e - c
+
+    if lo > hi:
+        return []
+    q_inf = ((e - 1) * 2 * m) // ((e + 1) * D)
+
+    def first_true(a, b, pred):          # smallest x in [a, b] with pred, else b + 1
+        while a < b:
+            mid = (a + b) // 2
+            if pred(mid):
+                b = mid
+            else:
+                a = mid + 1
+        return a if pred(a) else a + 1
+
+    far = max(hi, q_inf + 1) + 1
+    x1 = first_true(0, q_inf, lambda x: dP(x) >= 0)           # P' rising here
+    x2 = first_true(q_inf + 1, far, lambda x: dP(x) < 0)      # P' falling here
+    cuts = {lo, hi, q_inf, q_inf + 1, x1 - 1, x1, x2 - 1, x2}
+    pts = sorted(x for x in cuts if lo <= x <= hi)
+    roots = set()
+    for x in pts:
+        if P(x) == 0:
+            roots.add(x)
+    for x, y in zip(pts, pts[1:]):
+        px, py = P(x), P(y)
+        if px == 0 or py == 0 or (px > 0) == (py > 0):
+            continue
+        up = py > px
+        a, b = x, y
+        while b - a > 1:
+            mid = (a + b) // 2
+            pm = P(mid)
+            if pm == 0:
+                roots.add(mid)
+                break
+            if (pm < 0) == up:
+                a = mid
+            else:
+                b = mid
+    return sorted(roots)
+
+
 def factorize(n):
     f = {}
```

3.2 Issue 8: the rule that does not terminate

The added condition tests only `hi`, which for an open component uses the limit $p/(p-1)$ and so does not change as the exponent bound rises. On the all-open path the condition therefore holds forever. Measured effect: an open exponent driven past $2 \cdot 400$ after 256,506 states.

```
--- qp.v14.py
+++ qp.v16.py
@@ -77,7 +77,8 @@

 class Search:
-    def __init__(self, c=1, q_limit=10**7, log=None, refine_width=64):
+    def __init__(self, c=1, q_limit=10**7, log=None, refine_width=64, refine_qmax=5000):
         self.c = c
         self.refine_width = refine_width
+        self.refine_qmax = refine_qmax
         self.q_limit = q_limit   # max primes enumerated at one node before flagging
         self.nodes = 0
@@ -166,4 +167,11 @@
```

```
        if r == 1 and not opens:
            self.last_prime(comps, lo, nmin, last)
+           return
+       if r >= 2 and opens and r * hi / (2 - hi) > self.refine_qmax:
+           # hi is so close to 2 that the next prime ranges over a huge interval,
+           # and every choice would re-refine the same open exponents. Refine
+           # them once, here, instead: most exponent choices pull hi well below
+           # 2 and the branch dies at once. Ordering only, not a prune.
+           self.refine(comps, opens, r)
            return
        if r == 1:
```

3.3 Issue 8: the fix

The condition now also tests the lower end of the admissible range, which does rise with the exponents, so it eventually fails. Measured effect: 10^6 states on the same subtree with the deepest open exponent never exceeding $2 \cdot 15$.

```
--- qp.v16.py
+++ qp.v17.py
@@ -168,11 +168,17 @@
            self.last_prime(comps, lo, nmin, last)
            return
-       if r >= 2 and opens and r * hi / (2 - hi) > self.refine_qmax:
-           # hi is so close to 2 that the next prime ranges over a huge interval,
-           # and every choice would re-refine the same open exponents. Refine
-           # them once, here, instead: most exponent choices pull hi well below
-           # 2 and the branch dies at once. Ordering only, not a prune.
-           self.refine(comps, opens, r)
-           return
+       if r >= 2 and opens:
+           # When hi is very close to 2 the next prime ranges over a huge
+           # interval, and every choice would re-refine the same open exponents.
+           # Refine them once, here, instead: most exponent choices pull hi well
+           # below 2 and the branch dies at once. hi alone does not shrink as
+           # exponents grow, so the test also uses the lower end q_lo, which
+           # rises towards q_hi / r; the condition therefore fails eventually
+           # and refinement terminates. Ordering only, not a prune.
+           q_hi = r * hi / (2 - hi)
+           q_lo = lo / (2 - lo + F(c, nmin))
+           if q_hi > self.refine_qmax and q_hi > 4 * r * q_lo:
+               self.refine(comps, opens, r)
+               return
        if r == 1:
            # The last prime lies in (lo/(2-lo), 2/(2-hi)), a range whose width
```